



Port Said University
Faculty Of Engineering
Department Of Electrical Engineering



Smart Parking System Platform using Artificial Intelligence and Computer Vision

ركني – Rakeny

Graduation Project 2024



Supervisor	Dr. Emad El-Sayed
Team Members	Hala Essam Dawoud Nour Sherif Abuelenin Rawan Elsayed Zangir Rovan Wael Gowayed
Department Name	Department of Computer and Control Engineering
Date of Submission	July 2024

Abstract

Cities around the world are facing significant challenges posed by growing populations and increasing social unrest [1]. In Cairo, for example—arguably one of the largest cities in Africa—traffic congestion, high fuel consumption rates, and limited parking availability are major issues that citizens encounter during their daily commutes. These persistent problems necessitate the development of efficient solutions to improve urban quality of life and mobility.

One of the critical concerns among urban citizens is the difficulty in finding available parking spots in overcrowded areas. This issue is particularly acute during rush hours when city centers are saturated with vehicles, and the hunt for a vacant parking space can take anywhere from 5 to 30 minutes, leading to significant fuel wastage and increased traffic congestion [2]. This time-consuming and frustrating search process underscores the need for a smart and efficient parking solution.

Our platform, Rakeny for stress-free parking, addresses this challenge through an innovative computer vision-based smart parking system designed for urban environments and parking garages. The system harnesses the power of machine learning (ML), artificial intelligence (AI), and computer vision technologies to provide real-time aerial imagery of streets and parking areas. These images are processed to identify and extract information on available parking spots, offering a comprehensive and up-to-date view of parking availability.

Rakeny goes beyond mere detection by integrating a sophisticated guidance system that assists drivers in navigating to the nearest available parking spots relative to their destinations, both in outdoor and indoor settings. This feature not only reduces the time and effort spent on finding parking but also helps in lowering fuel consumption and minimizing traffic disruptions. Additionally, Rakeny includes a functionality to remind drivers of their parking location, thereby eliminating the common hassle of forgetting where they parked their vehicle.

In summary, Rakeny aims to revolutionize the parking experience by making it more efficient and less stressful. By leveraging cutting-edge technologies, our platform seeks to mitigate the parking challenges faced by urban drivers, contributing to smoother traffic flow, reduced environmental impact, and enhanced overall quality of life in densely populated cities.

Acknowledgments

The successful completion of our graduation project would not have been possible without the invaluable support and contributions of several individuals and groups.

First and foremost, we would like to express our sincere gratitude to our esteemed supervisor, Dr. Emad El-Sayed.

His guidance and mentorship have been instrumental throughout this project. Dr. El-Sayed's deep knowledge of the field and his ability to provide insightful feedback were crucial in shaping the direction of our research and ensuring its academic rigor. His encouragement and unwavering belief in our abilities helped us navigate challenges and persevere during difficult moments.

We would also like to extend our heartfelt appreciation to our families for their unwavering support throughout this entire journey.

Their patience and understanding during countless hours of research and writing were invaluable. Their words of encouragement during moments of doubt and their celebration of milestones kept us motivated and focused on achieving our goals.

In closing, we would like to express our gratitude to all those who have played a role in bringing this project to fruition.

Thank you all.

Table of Contents

ABSTRACT	I
ACKNOWLEDGMENTS.....	II
TABLE OF CONTENTS.....	III
LIST OF FIGURES.....	V
LIST OF TABLES	VI
CHAPTER 1: INTRODUCTION.....	1
1.1 BACKGROUND: THE GROWING CHALLENGE OF URBAN PARKING MANAGEMENT.....	1
1.2 MOTIVATION: THE NEED FOR SMART PARKING SOLUTIONS	2
1.3 OBJECTIVES: ADDRESSING URBAN PARKING CHALLENGES THROUGH RAKENY	3
1.4 SCOPE: RAKENY'S FUNCTIONALITY AND LIMITATIONS.....	5
1.5 TARGETED CUSTOMERS: RAKENY'S CUSTOMERS	7
CHAPTER 2: RAKENY BUSINESS MODEL	8
2.1 INTRODUCTION	8
2.2 KEY PARTNERS	9
2.3 KEY ACTIVITIES.....	9
2.4 KEY RESOURCES.....	9
2.5 VALUE PROPOSITIONS	10
2.6 CUSTOMER RELATIONSHIPS	10
2.7 CHANNELS	10
2.8 CUSTOMER SEGMENTS.....	10
2.9 COST STRUCTURE	11
2.10 REVENUE STREAMS	11
2.11 CONCLUSION.....	11
CHAPTER 3: LITERATURE REVIEW	12
3.1 EXISTING SOLUTIONS	12
3.2 GAPS IN CURRENT SOLUTIONS.....	14
3.3 THE NEED FOR RAKENY: A SMARTER APPROACH TO PARKING	16
CHAPTER 4: SYSTEM DESIGN.....	17
4.1 OVERALL ARCHITECTURE.....	17
4.2 AI MODEL	22
4.3 ADMIN INTERFACE.....	26
4.4 MOBILE APPLICATION.....	29
4.5 INTEGRATION.....	34
CHAPTER 5: IMPLEMENTATION	36
5.1 DEVELOPMENT ENVIRONMENT	36
5.2 AI MODEL IMPLEMENTATION.....	38
5.3 ADMIN INTERFACE DEVELOPMENT	45
5.4 MOBILE APPLICATION DEVELOPMENT	57
5.5 SYSTEM INTEGRATION	72
CHAPTER 6: TESTING AND EVALUATION	76
6.1 TESTING STRATEGY	76
6.2 TEST CASES AND RESULTS.....	77
6.3 PERFORMANCE EVALUATION	78

6.4 DISCUSSION	79
CHAPTER 7: CONCLUSION AND FUTURE WORK.....	80
7.1 SUMMARY OF FINDINGS	80
7.2 CONTRIBUTIONS	82
7.3 LIMITATIONS	84
7.4 FUTURE WORK	85
REFERENCES	I
APPENDICES.....	II
APPENDIX A: ADDITIONAL TECHNICAL DETAILS.....	II
APPENDIX B: USER MANUALS FOR THE MOBILE APP AND ADMIN INTERFACE.	IV
APPENDIX C: SURVEY/QUESTIONNAIRE TEMPLATES USED FOR USER FEEDBACK.....	VI
GLOSSARY	VII

List of Figures

FIGURE 1: RAKENY'S LIFE CYCLE	5
FIGURE 2: RAKENY'S BUSINESS MODEL	8
FIGURE 3: RAKENY'S OVERALL STRUCTURE	17
FIGURE 4: VEHICLES AND PARKING SPACE DETECTION.....	40
FIGURE 5: CAMERA CALIBRATION PROCESS	41
FIGURE 6: ADMIN INTERFACE LOGIN PAGE	48
FIGURE 7: ADMIN INTERFACE LOGIN PAGE 2	48
FIGURE 8: ADMIN INTERFACE -SUPER ADMIN- DASHBOARD.....	49
FIGURE 9: ADMIN INTERFACE -SUB ADMIN- DASHBOARD	49
FIGURE 10: ADMIN INTERFACE FEED (ADD CAMERA PROCESS).....	50
FIGURE 11: ADMIN INTERFACE FEED (ADD BOUNDING BOX PROCESS).....	50
FIGURE 12: ADMIN INTERFACE FEED (TESTING CAMERA PROCESS).....	51
FIGURE 13: ADMIN INTERFACE FEED (CALIBRATION PROCESS)	51
FIGURE 14: ADMIN INTERFACE CONTROL ROOM.....	52
FIGURE 15: ADMIN INTERFACE REPORTS.....	53
FIGURE 16: ADMIN INTERFACE PROFILE PAGE	54
FIGURE 17: ADMIN INTERFACE SETTINGS 1.....	55
FIGURE 18: ADMIN INTERFACE SETTINGS 2.....	55
FIGURE 19: ADMIN INTERFACE SETTINGS 3.....	56
FIGURE 20: USER INTERFACE STORY BOARD.....	58
FIGURE 21: MOBILE APPLICATION ONBOARDING PAGE	59
FIGURE 22: MOBILE APPLICATION AUTHENTICATION PAGE	60
FIGURE 23: MOBILE APPLICATION HOME PAGE	61
FIGURE 24: MOBILE APPLICATION NAVIGATION PAGE	62
FIGURE 25: MOBILE APPLICATION ACCOUNT PAGE.....	63
FIGURE 26: MOBILE APPLICATION HISTORY PAGE	64
FIGURE 27: MOBILE APPLICATION SETTINGS PAGES	65
FIGURE 28: MOBILE APPLICATION EDIT USER DATA PAGE.....	66

List of Tables

TABLE 1: KEY ISSUES IN URBAN PARKING MANAGEMENT..... 3

TABLE 2: ADMIN INTERFACE USER ROLES AND PERMISSIONS..... 27

TABLE 3: TECHNOLOGIES AND LIBRARIES USED IN ARTIFICIAL INTELLIGENCE MODEL..... 44

TABLE 4: HTTP ENDPOINT SUMMARY TABLE..... 74

Chapter 1: Introduction

1.1 Background: The Growing Challenge of Urban Parking Management

The ever-expanding sprawl of urban landscapes presents a multitude of challenges for city planners, and one of the most pressing issues is parking management. As populations in major cities continue to surge, the demand for parking spaces outpaces the ability of existing infrastructure to keep pace. This imbalance creates a domino effect of problems that impact not only drivers but also the overall well-being of urban environments.

- **Population Growth**

The exponential rise in urban populations is a primary driver of the parking predicament. As more people flock to cities, the number of vehicles on the road naturally increases. This surge in vehicle ownership presents a significant challenge for urban planning, as current parking infrastructure often struggles to accommodate the growing demand.

- **Traffic Congestion**

The limited availability of parking spaces directly contributes to traffic congestion. Drivers forced to spend extended periods searching for parking spots contribute to clogged roadways, leading to increased travel times, frustration, and driver inefficiency. Furthermore, congested roads can create bottlenecks that further exacerbate the difficulty of finding parking.

- **Limited Parking Availability**

Existing parking infrastructure, whether on-street parking or traditional parking garages, often falls short of meeting the growing demand. This scarcity can lead to a "circling game" where drivers waste valuable time and fuel searching for vacant spots, further compounding traffic congestion and environmental concerns.

These interconnected issues highlight the urgent need for innovative and efficient solutions to manage urban parking effectively. The development of smart parking systems, like Rakeny, offers a promising approach to address these challenges and create a more sustainable and user-friendly urban environment.

1.2 Motivation: The Need for Smart Parking Solutions

The current state of urban parking management presents a significant burden on drivers, the environment, and city infrastructure. This necessitates the development of smart parking solutions like Rakeny, which can address the following key challenges:

- **Time-Consuming Parking Searches**

The traditional method of searching for parking often involves aimlessly cruising through streets or circling parking lots, wasting valuable time and causing frustration for drivers. Studies have shown that drivers can spend anywhere from 5 to 30 minutes searching for parking, leading to significant productivity losses and increased stress levels. [2]

This inefficiency not only impacts individual drivers but also contributes to overall traffic congestion as frustrated motorists clog roadways.

- **Fuel Wastage**

The time spent searching for parking translates directly to wasted fuel consumption. Vehicles circling for parking spots contribute significantly to increased emissions, further deteriorating air quality in urban environments.

A study found that cruising for parking can account for up to 30% of urban traffic congestion, directly impacting fuel efficiency and environmental sustainability. [1]

- **Traffic Disruptions**

The inefficiency of the current parking system creates a ripple effect that disrupts traffic flow. Drivers searching for parking contribute to congestion, slowing down overall traffic movement.

Additionally, vehicles double-parking or blocking intersections while searching for spots further exacerbating the problem. This creates a vicious cycle where the difficulty of finding parking contributes to traffic congestion, which in turn makes finding parking even more challenging.

Smart parking systems like Rakeny have the potential to revolutionize urban parking management by addressing these issues. By providing real-time data on parking availability and guiding drivers to vacant spots, Rakeny aims to minimize wasted time, reduce fuel consumption, and ultimately improve traffic flow within urban environments.

1.3 Objectives: Addressing Urban Parking Challenges Through Rakeny

The growing demand for parking in urban environments necessitates innovative solutions that optimize efficiency and enhance the overall driver experience.

Table 1: Key Issues in Urban Parking Management

Issue	Description	Impact
Inefficient Parking Space Utilization	A significant portion of available parking spaces remain unoccupied, while drivers circle searching for spots, wasting time and fuel.	Increased traffic congestion, wasted fuel and emissions, driver frustration.
Limited Parking Availability	Demand for parking often outstrips supply, particularly in densely populated areas, leading to competition for scarce spaces.	Difficulty finding parking, increased time spent searching, potential for illegal parking.
Lack of Real-time Information	Drivers often lack real-time information about available parking, leading to inefficient search patterns and wasted time.	Increased traffic congestion, wasted fuel and emissions, driver frustration.
Ineffective Pricing and Enforcement	Inconsistent or unclear parking regulations and enforcement can lead to confusion and improper parking behavior.	Inefficient use of parking spaces, potential safety hazards, difficulty managing parking turnover.
Limited Accessibility for People with Disabilities	A lack of designated and readily available parking spaces for people with disabilities creates challenges for accessibility.	Difficulty for people with disabilities to access destinations, hinders social inclusion.
Environmental Impact	Cruising for parking contributes to increased traffic congestion, leading to higher fuel consumption and emissions.	Negative impact on air quality, contributes to climate change.

Rakeny project aims to address these challenges through the development and implementation of Rakeny, a smart parking system with the following key objectives:

- **Develop a Real-Time Parking Availability System**

Rakeny will leverage the power of Artificial Intelligence (AI) and computer vision technology to create a real-time parking availability system. By deploying strategically placed cameras or utilizing existing infrastructure with camera capabilities, the system will capture real-time aerial or street-level imagery of parking areas.

This visual data will then be processed using advanced AI algorithms to accurately identify and classify vacant parking spaces. This real-time information will be readily accessible to drivers through a user-friendly mobile application, eliminating the need for time-consuming searches.

- **Optimize Parking Efficiency**

By providing real-time data on parking availability, Rakeny aims to significantly reduce the time drivers spend searching for parking spots. This not only improves individual driver experience but also contributes to a more efficient flow of traffic throughout urban areas.

Reduced search times will lead to lower fuel consumption, minimizing environmental impact and promoting sustainability. Additionally, Rakeny can potentially integrate with existing traffic management systems to optimize traffic flow by directing drivers towards areas with higher parking availability.

- **Enhance Driver Experience**

Rakeny is designed to offer a user-friendly and convenient platform for managing parking needs. The mobile application will provide drivers with real-time information on available parking locations, including navigation assistance to guide them directly to the nearest vacant spot.

This eliminates the frustration and wasted time associated with traditional parking searches. Furthermore, Rakeny can incorporate features like parking reminders to help drivers avoid the common hassle of forgetting where they parked their vehicle. By prioritizing user convenience and offering a seamless parking experience, Rakeny aims to improve the overall satisfaction of drivers in urban environments.

1.4 Scope: Rakeny's Functionality and Limitations

Rakeny's development is focused on addressing the specific challenges of parking management in urban environments.

1.4.1 Rakeny's Main Functionality

- **Target Areas**

Rakeny will primarily target densely populated urban areas where parking scarcity is a significant concern. This includes city centers, business districts, and residential neighborhoods with limited on-street parking options. While the project focuses on urban settings, the core functionalities may be adaptable to other environments in the future.

- **AI-Powered Functionality**

The core functionality of Rakeny revolves around its AI-powered parking detection and guidance system. The system will utilize computer vision to capture real-time images of parking areas, either through strategically placed cameras or by leveraging existing infrastructure with camera capabilities (e.g., traffic cameras).

Advanced AI algorithms will then analyze these images to accurately identify and classify vacant parking spaces. This information will be translated into real-time data accessible through the user-friendly mobile application. Additionally, the AI will integrate with navigation systems to guide drivers efficiently to their chosen parking spots.

- **User-Centric Design**

Rakeny prioritizes user convenience by offering a user-friendly mobile application as the primary interface. The application will display real-time parking availability data, including location information and navigation assistance.

Beyond basic parking search functionalities, Rakeny may explore additional features that enhance user experience. This could include functionalities like:

- **Payment Integration:** Incorporating secure payment options within the app could allow users to pay for parking fees directly, eliminating the need for separate parking meters or kiosks.
- **Parking History:** Maintaining a history of past parking locations within the app can assist users in easily recalling where they parked their vehicle.
- **Real-time Updates:** The system can provide real-time updates on parking availability changes, allowing users to adjust their parking strategy based on dynamic data.

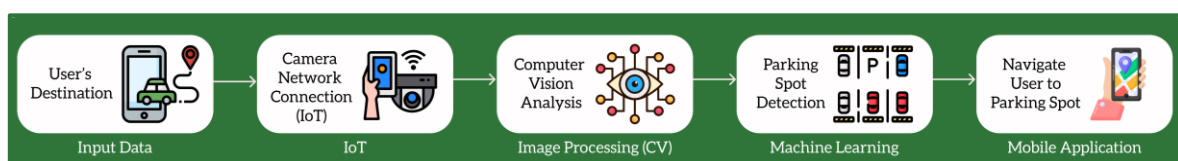


Figure 1: Rakeny's Life Cycle

1.4.2 Rakeny's Main Limitations

It is important to acknowledge the limitations of the project during its development phase.

- **Infrastructure Dependence**

Rakeny's effectiveness relies on the availability of camera infrastructure within the target areas. The project may need to explore alternative data acquisition methods if existing camera infrastructure is insufficient.

- **Data Security**

Ensuring the security and privacy of user data collected through the mobile application is paramount. Robust security protocols and data encryption methods will be implemented to protect user information.

- **Scalability**

While the initial focus is on urban environments, future iterations of Rakeny may explore scalability to accommodate different parking scenarios (e.g., shopping mall parking lots, university parking structures).

By acknowledging these limitations and outlining potential solutions, the project demonstrates a comprehensive understanding of the challenges and opportunities associated with Rakeny's development.

1.5 Targeted Customers: Rakeny's Customers

Rakeny's innovative smart parking system caters to two distinct customer segments, each with unique needs and benefits:

1.5.1 Governmental Agencies

Rakeny offers a valuable solution for city governments and urban planning authorities struggling with growing parking challenges. The system leverages existing public infrastructure to deliver:

- **Real-time Parking Availability Data:** Utilize existing camera feeds to provide citizens with real-time information on available parking spots, reducing search times and traffic congestion.
- **Reduced Infrastructure Investment:** Rakeny leverages existing infrastructure, minimizing the need for additional investment in dedicated parking sensors.
- **Enhanced Urban Mobility:** By optimizing parking efficiency, Rakeny contributes to smoother traffic flow and improved urban mobility.
- **Data-Driven Decision Making:** Rakeny provides valuable data insights that can inform future parking infrastructure development and management strategies.

1.5.2 Commercial Parking Facility Owners

Rakeny caters to private parking facilities, offering a comprehensive solution to optimize operations and enhance customer experience:

- **Improved Parking Management:** Gain real-time data on parking availability and utilization to optimize parking allocation strategies.
- **Data-Driven Revenue Optimization:** Identify peak usage times and popular parking zones to potentially adjust pricing structures and maximize revenue.
- **Enhanced Customer Satisfaction:** Provide real-time parking availability information and streamline the parking experience for customers.
- **Marketing and Customer Engagement:** Leverage a dedicated mobile application to directly engage with customers, offer promotions, and collect valuable feedback.

By addressing the specific needs of both governmental agencies and commercial parking facilities, Rakeny positions itself as a comprehensive and valuable smart parking solution for urban environments.

Chapter 2: Rakeny Business Model

2.1 Introduction

This chapter provides a comprehensive analysis of the business model for Rakeny, a smart parking system platform. The analysis covers key components of the business model, including key partners, activities, resources, value propositions, customer relationships, channels, customer segments, cost structure, and revenue streams.

The goal is to elucidate how Rakeny aims to create, deliver, and capture value in the urban mobility and parking management sector.

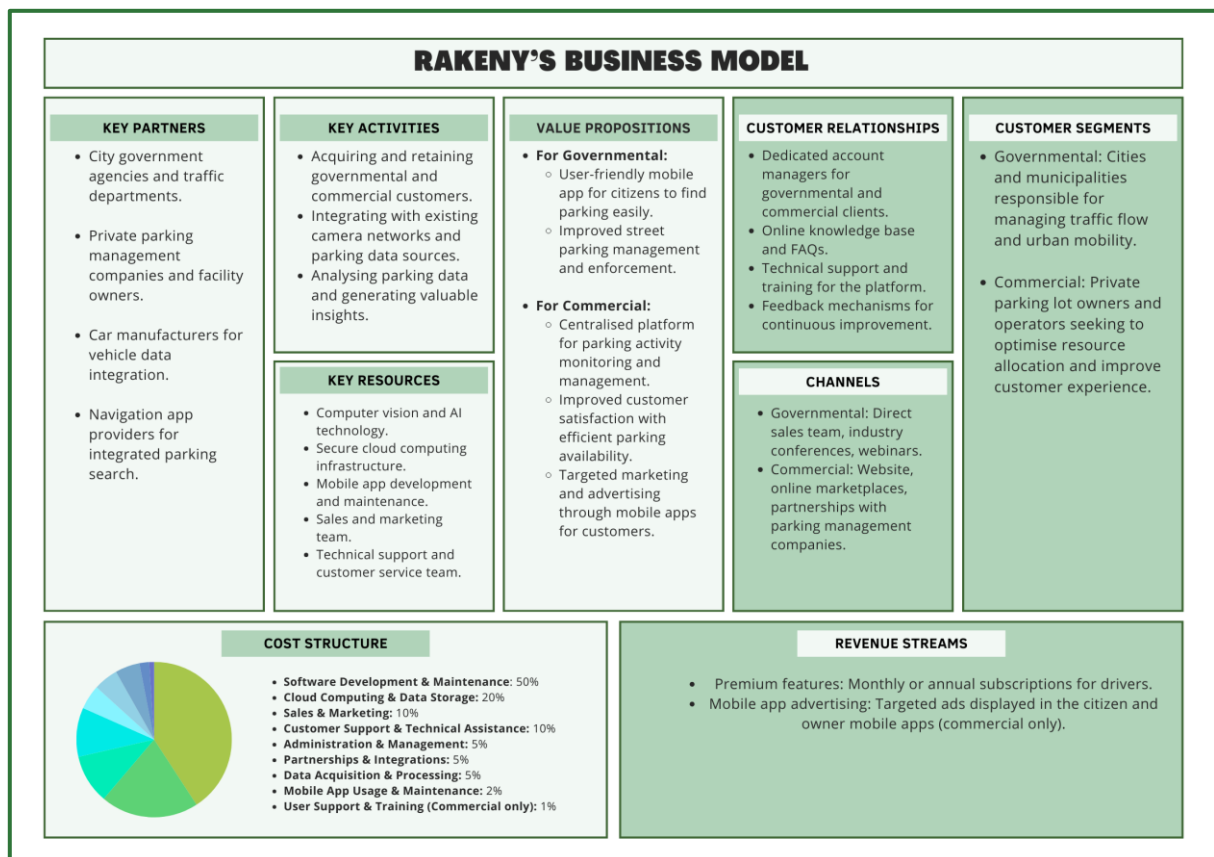


Figure 2: Rakeny's Business Model

2.2 Key Partners

Rakeny's success hinges on strategic partnerships that provide essential support and resources. The key partners include:

- **City Government Agencies and Traffic Departments:** Collaborations with municipal bodies facilitate integration with existing urban infrastructure and ensure regulatory compliance.
- **Private Parking Management Companies and Facility Owners:** These partnerships enable access to a network of parking spaces and data sources.
- **Car Manufacturers:** Integration with vehicle data systems enhances the platform's capability to provide real-time parking information.
- **Navigation App Providers:** Partnerships with providers such as Google Maps or Waze enhance the user experience by incorporating parking search functionalities.

2.3 Key Activities

The primary activities essential to Rakeny's operation include:

- **Acquiring and Retaining Customers:** Focused efforts on both governmental and commercial clients ensure a stable and expanding user base.
- **Integrating with Existing Systems:** Seamless integration with existing camera networks and parking data sources is crucial for real-time data accuracy.
- **Analyzing Parking Data:** Continuous data analysis generates valuable insights that help improve the system and user experience.

2.4 Key Resources

Rakeny's core resources are pivotal in delivering its value propositions:

- **Computer Vision and AI Technology:** These technologies form the backbone of the platform, enabling the identification and monitoring of parking spaces.
- **Secure Cloud Computing Infrastructure:** Ensures data security and scalable performance.
- **Mobile App Development and Maintenance:** A dedicated team for ongoing app development ensures user-friendly interfaces and reliable performance.
- **Sales and Marketing Team:** Vital for customer acquisition and retention.
- **Technical Support and Customer Service Team:** Provides necessary support to users and ensures smooth operation.

2.5 Value Propositions

Rakeny offers distinct value propositions tailored to different customer segments:

- **For Governmental Clients:**
 - **User-friendly Mobile App:** Allows citizens to find parking easily, improving daily commute experiences.
 - **Improved Street Parking Management and Enforcement:** Enhances urban traffic flow and reduces congestion.
- **For Commercial Clients:**
 - **Centralized Monitoring Platform:** Facilitates efficient parking activity management.
 - **Enhanced Customer Satisfaction:** Ensures better parking availability and targeted marketing through mobile apps.

2.6 Customer Relationships

Maintaining strong customer relationships is critical for Rakeny:

- **Dedicated Account Managers:** Personalized service for governmental and commercial clients.
- **Online Knowledge Base and FAQs:** Provides users with self-help resources.
- **Technical Support and Training:** Ensures users can fully leverage the platform's capabilities.
- **Feedback Mechanisms:** Continuous improvement through customer feedback.

2.7 Channels

Rakeny utilizes multiple channels to reach and serve its customers:

- **Governmental:** Direct sales team, industry conferences, and webinars facilitate engagement with public sector clients.
- **Commercial:** Online marketplace, website, and partnerships with parking management companies expand the platform's reach.

2.8 Customer Segments

Rakeny targets two primary customer segments:

- **Governmental Clients:** Cities and municipalities responsible for managing traffic flow and urban mobility.
- **Commercial Clients:** Private parking lot owners and operators seeking to optimize resource allocation and improve customer experience.

2.9 Cost Structure

The cost structure of Rakeny highlights the significant areas of expenditure:

- **Software Development and Maintenance (50%):** Major investment in technology development.
- **Cloud Computing and Data Storage (20%):** Ensures scalable and secure data handling.
- **Sales and Marketing (10%):** Essential for customer acquisition.
- **Customer Support and Technical Assistance (10%):** Critical for maintaining high user satisfaction.
- **Administration and Management (5%):** Operational costs.
- **Partnerships and Integrations (5%):** Developing and maintaining strategic partnerships.
- **Data Acquisition and Processing (5%):** Ensures the quality and accuracy of parking data.
- **Mobile App Usage and Maintenance (2%):** Regular updates and feature enhancements.
- **User Support and Training (1%):** Specifically for commercial clients.

2.10 Revenue Streams

Rakeny generates revenue through:

- **Premium Features:** Monthly or annual subscriptions for drivers, providing enhanced functionalities.
- **Mobile App Advertising:** Targeted ads displayed in both citizen and owner mobile apps, offering a commercial opportunity for partners.

2.11 Conclusion

The business model of Rakeny is designed to leverage cutting-edge technology and strategic partnerships to address urban parking challenges. By focusing on providing value to both governmental and commercial clients, Rakeny aims to streamline parking management, reduce traffic congestion, and improve the overall urban mobility experience. The detailed cost structure and diverse revenue streams ensure the platform's financial sustainability and potential for growth.

Chapter 3: Literature Review

3.1 Existing Solutions

The ever-growing number of vehicles in urban environments has strained traditional parking management systems, leading to frustration for drivers and inefficiencies for parking facility operators. To address this challenge, various technologies have emerged, each with its own strengths and weaknesses.

3.1.1 Manual Systems:

- **Parking Meters**

These are the most common parking management tools. Drivers physically pay for parking time using coins or credit cards, often leading to inconvenience and delays, especially with limited change or malfunctioning meters.

Additionally, parking meter systems require ongoing maintenance and cash collection, adding to operational costs.

- **Signage and Attendants**

Limited parking availability is often indicated by signage, such as "No Parking" zones or designated loading areas. While helpful, these signs are static and may not reflect real-time changes in parking availability.

Additionally, parking attendants, while providing a human touch, may not have immediate updates on available spots, especially in large parking facilities.

3.1.2 Sensor-Based Systems:

These systems offer a step towards automation but have their own limitations:

- **Magnetic or Ultrasonic Sensors**

These sensors are embedded in the pavement of individual parking spaces. They detect the presence or absence of a vehicle by sensing the magnetic field disruption caused by a car's metal frame (magnetic) or by emitting ultrasonic waves that bounce off an occupying vehicle (ultrasonic).

While offering real-time data, these systems require significant infrastructure investment for installation and maintenance. Additionally, sensors embedded in the pavement can be susceptible to damage from weather or wear and tear, leading to malfunctioning systems and inaccurate data.

- **License Plate Recognition (LPR) Systems**

LPR systems use strategically placed cameras to capture license plates of vehicles entering and exiting parking facilities. The captured data is then used to track vehicle movement and calculate parking fees based on entry and exit times.

While offering automated billing capabilities, LPR systems can be expensive to install and maintain, requiring high-resolution cameras and complex software. Additionally, their effectiveness can be hampered by factors like poor lighting conditions, dirty or obscured license plates, or camera angles that struggle to capture all plates clearly.

3.1.3 Computer Vision-Based Systems:

Computer vision techniques have been applied to detect and monitor available parking spaces through trained models. [3]

A similar project applied image processing and enhancement to keep track of the cars entering and leaving the parking using license identification technology. [4]

The license of these plates are taken account of in real-time and this data is used to generate the parking bills given the in-time and out-time of the vehicle.

Another similar project proposes to design a more adaptable and affordable smart parking system via distributed cameras, edge computing, data analytics, and advanced deep learning algorithms. [5]

Specifically, they deploy cameras with zoom-lens and motorized head to capture license plate numbers by tracking the vehicles when they enter or leave the parking lot; cameras with wide angle fish-eye lenses will monitor the large parking lot via our custom designed deep neural network.

Which allows them to detect when a car enters the parking space, the location of the parking spot, and precisely charge the parking fee and associate this with the license plate number.

3.2 Gaps in Current Solutions

The ever-increasing demand for parking in crowded areas exposes the shortcomings of traditional parking management systems. These solutions, while offering some functionality, struggle to keep pace with the growing complexities of parking.

3.2.1 Inefficiency and Inaccuracy

- **Manual Systems**

Parking meters and attendants, while familiar, create bottlenecks and delays. The physical interaction required for payment with meters can be inconvenient, especially during peak hours. Additionally, limited meter availability or malfunctioning equipment can further disrupt the parking experience.

Attendants, although helpful, may not have access to real-time data on parking availability, leading drivers on a frustrating search for non-existent open spaces. Static signage, another staple of manual systems, often lacks real-time updates, causing wasted time and driver frustration as they search for spots based on potentially outdated information.

- **Sensor-Based Systems**

While offering a step towards automation, sensor-based systems have their own set of limitations that impact user experience. Sensor failure or malfunction can provide inaccurate data, leading drivers on a wild goose chase for unavailable spots. This not only wastes time and fuel but also contributes to driver frustration.

Furthermore, sensor-based systems typically only detect presence or absence, lacking the ability to capture details like parking space size or type (e.g., handicapped accessible). This can be inconvenient for drivers with specific parking needs who may waste time searching for a suitable spot that isn't identified by the sensors.

3.2.2 Limited Scalability and Cost

- **Sensor-Based Systems:**

Installing and maintaining a network of sensors in every parking space can be a significant upfront investment, especially for large parking facilities or densely populated areas. The cost of purchasing, installing, and maintaining the sensors can be a major barrier to entry for municipalities or private parking operators with limited budgets.

This limitation hinders the scalability of sensor-based solutions, restricting their application to areas where the high upfront cost can be justified.

- **LPR Systems**

The high cost of high-resolution cameras, complex software, and data management infrastructure can limit the adoption of LPR systems in smaller parking facilities or budget-conscious areas. Like sensor-based systems, the initial investment required for LPR can be a significant barrier to entry, especially for areas with lower parking volumes or those seeking a more cost-effective solution.

3.2.3 Data Limitations and Privacy Concerns

- **Manual Systems**

These offer minimal data collection beyond basic occupancy information. This lack of comprehensive data hinders efforts to analyze parking usage patterns and optimize parking management strategies. Without insights into parking trends and behavior, it's difficult for municipalities or parking facility operators to make data-driven decisions that improve efficiency and user experience.

- **LPR Systems**

While offering some data on vehicle movement through license plate capture, LPR systems raise privacy concerns regarding the collection and storage of this personal data. Drivers may be hesitant to utilize parking facilities that employ LPR due to potential privacy violations, especially if data retention policies are unclear or lack transparency. This can limit the effectiveness of LPR systems, as some drivers may choose alternative parking options to avoid privacy concerns.

3.3 The Need for Rakeny: A Smarter Approach to Parking

The shortcomings of existing parking solutions expose a critical gap in today's crowded environments. These traditional methods struggle to keep pace with the growing demand for efficient and user-friendly parking. Here's where Rakeny steps in, offering a comprehensive solution built on the power of computer vision technology:

- **Real-Time Data, Optimized Efficiency**

Rakeny delivers real-time parking availability information, eliminating the time wasted circling for spots and minimizing traffic congestion. This not only enhances the driver experience but also contributes to a smoother flow of traffic throughout crowded areas.

- **Cost-Effective Scalability**

By leveraging existing camera infrastructure or readily available cameras, Rakeny reduces installation costs compared to sensor-based systems. This translates into a highly scalable solution, seamlessly adaptable to diverse environments – from small private parking lots to expansive facilities in densely populated areas.

- **Data-Driven Insights, Privacy-Preserving**

Rakeny focuses on vehicle detection, extracting valuable data without capturing license plates. This anonymized data empowers municipalities and parking facility operators to analyze parking patterns and optimize management strategies, all while respecting user privacy.

By leveraging these insights, informed decisions can be made to improve parking allocation, identify areas for enhancement, and ultimately create a more efficient and user-friendly parking experience in crowded environments.

Rakeny's innovative approach bridges the gap between outdated methods and the demands of modern parking management. It offers a win-win scenario for all stakeholders:

- **Drivers:** Enjoy the convenience of real-time parking information, reducing search times and frustration.

- **Municipalities and Urban Planners:** Gain data-driven insights to optimize parking allocation, reduce traffic congestion, and enhance overall mobility.

- **Parking Facility Operators:** Benefit from cost-effective management with real-time data to optimize pricing strategies and improve customer satisfaction.

- **The Environment:** Reduced traffic congestion and improved efficiency contribute to lower emissions, fostering a cleaner environment in crowded areas.

Rakeny represents a paradigm shift in parking management, offering a smarter and more sustainable approach for the future of crowded environments.

Chapter 4: System Design

4.1 Overall Architecture

Rakeny's architecture is designed for scalability, efficiency, and user-friendliness, addressing the challenges of parking management in crowded environments. The system leverages a modular approach, integrating distinct components to deliver a comprehensive solution. Each component plays a crucial role in ensuring the seamless operation of the system, from real-time vehicle detection to user interaction and data management.

Rakeny's architecture is built on a foundation of advanced technologies and best practices in system design. The integration of edge AI for real-time data processing, secure cloud storage for scalable data management, and intuitive user interfaces for both administrators and end-users ensures a holistic approach to parking management.

The system is designed to be adaptable to various environments, capable of leveraging existing infrastructure, and scalable to accommodate growing demands. This flexibility makes Rakeny a versatile solution that can be implemented in diverse urban settings, from small parking lots to large multi-level parking facilities.



Figure 3: Rakeny's Overall Structure

4.1.1. Computer Vision Model

YOLOv8 Detection Model Deployment

The core of Rakeny's edge AI capabilities is a YOLOv8 deep learning model, deployed on cameras or local servers. This model performs real-time object detection and classification, identifying vehicles and determining parking availability.

The analysis focuses on differentiating between occupied and vacant parking spaces, ensuring accurate and timely updates. The YOLOv8 model has been trained on a diverse dataset of vehicles, enhancing its ability to recognize different car types and sizes. The model's lightweight architecture allows for efficient deployment on edge devices, ensuring low latency in real-time applications.

Hardware Infrastructure

Rakeny capitalizes on existing camera infrastructure or readily available cameras installed in parking facilities or public spaces. This eliminates the need for significant upfront investment in new hardware, making the solution adaptable to diverse environments.

The edge devices are equipped with sufficient computational power to run the YOLOv8 model and perform real-time analysis, ensuring immediate feedback on parking space occupancy.

Camera Calibration Process

To ensure the accuracy of the computer vision model, a rigorous camera calibration process is employed. Calibration involves determining the camera's intrinsic parameters (focal length, optical center, and distortion coefficients) and extrinsic parameters (position and orientation in the 3D space).

This process typically uses a known calibration pattern, such as a checkerboard, placed at various angles and distances. The calibration ensures that the YOLOv8 model can accurately interpret the spatial dimensions and positions of detected vehicles within the parking area, leading to precise identification of parking spaces and vehicle sizes.

Empty Space Calculation and Labeling

The system employs sophisticated algorithms to process the vehicle detection data obtained from the YOLOv8 model. These algorithms analyze the scene to differentiate between occupied and vacant parking spaces, potentially utilizing image segmentation techniques.

By examining factors like vehicle location and footprint within the image, the algorithm can accurately identify empty spaces and assign labels for further processing.

Car Size Estimation

To provide additional user-centric features, Rakeny incorporates a method for estimating the size of detected vehicles. This estimation might leverage historical data on parked car sizes within the specific location or employ real-time image analysis techniques to approximate vehicle dimensions.

This feature helps in determining the suitability of parking spaces for different car types, enhancing user experience by guiding drivers towards compatible spots.

4.1.2 Streamlit

Admin Interface

Streamlit powers the web-based admin interface, enabling real-time monitoring and management of the parking system. This interface provides administrators with several key features:

- **Real-Time Map View**

A visual representation of the parking area is provided, with color-coded indicators for occupied and vacant spaces. This map view allows for efficient monitoring and quick identification of available parking spots.

- **Authorized Parking Zone Management**

Administrators can define and manage parking zones within the monitored area. This feature is valuable in areas with designated parking restrictions or permit-only zones, ensuring compliance and efficient space utilization.

- **Customizable Reports**

The system offers the ability to generate reports based on user-defined parameters. These reports provide valuable insights into parking usage patterns, occupancy rates over time, and other relevant metrics. This data can inform strategic decision-making for parking management and optimization.

4.1.3 FlutterFlow

Mobile Application

The Rakeny mobile application is developed using FlutterFlow, the mobile app offers real-time features for drivers, enhancing the parking experience:

- **Navigation System**

The app integrates a navigation system that guides users directly to available parking spaces, reducing search times and frustration. This not only improves the user experience but also contributes to a smoother flow of traffic within crowded areas.

- **Up-to-Date Parking Information**

Drivers have access to real-time information on parking availability, including the number of open spaces and their suitability based on the user's car type. This transparency empowers drivers to make informed decisions about parking and avoid unsuitable locations.

- **Spot Saver Functionality**

The system can be optionally integrated with a spot saver functionality. This feature allows users to reserve a parking space for a limited time, providing a layer of convenience and eliminating the anxiety of potentially losing a suitable spot upon arrival. Integration with payment systems can further enhance this functionality, allowing users to reserve and pay for a spot directly through the app.

4.1.4 MongoDB

Data Storage and Management

MongoDB serves as the primary data storage solution, handling information on users, parking spaces, and usage histories. The database's flexible schema supports the dynamic nature of the data, allowing for efficient storage and retrieval operations. MongoDB's scalability ensures that the system can handle a growing volume of data as the number of users and monitored parking spaces increases.

- **User Data**

The database stores user profiles, including details such as usernames, email addresses, car types, and parking history. This information is used to personalize the user experience and provide tailored recommendations.

- **Parking Space Data**

Information about each parking space, including its location, status (occupied or vacant), and suitability for different car types, is maintained in the database. This data is continuously updated based on real-time inputs from the YOLOv8 model and processed algorithms.

- **Historical Data**

MongoDB stores historical data on parking space occupancy and usage patterns. This data is valuable for generating reports and conducting analyses to improve the system's performance and optimize parking management strategies.

4.1.5 MongoDB Atlas Data API

Secure Data Communication

The MongoDB Atlas Data API facilitates secure and efficient communication between the database and front-end applications.

This API ensures real-time synchronization of parking data, maintaining consistent and up-to-date information across all user interfaces. The API's robust security features, including encryption and authentication, protect user data and ensure compliance with privacy regulations.

- **Real-Time Data Exchange**

The API enables real-time data exchange between the edge devices, cloud platform, and user interfaces. This ensures that drivers receive accurate and timely information about parking availability, enhancing the overall user experience.

- **Data Integration**

The API supports seamless integration with other components of the Rakeny system, allowing for efficient data flow and processing. This integration is crucial for maintaining the accuracy and responsiveness of the system.

4.1.6 System Integration and Feedback Loop

Seamless API Communication

The various components of the Rakeny system communicate through secure APIs (Application Programming Interfaces). These APIs ensure the smooth flow of data between the edge devices, cloud platform, and user interfaces. Real-time data exchange is crucial for maintaining accurate parking availability information and delivering a responsive user experience.

Continuous Improvement through Data Analysis

The anonymized data collected from the system offers a wealth of insights into parking usage patterns and user behavior. By leveraging machine learning and data analytics techniques, Rakeny can continuously improve its performance over time.

- **Model Refinement**

Data analysis can be used to identify potential shortcomings in the YOLOv8 model's object detection accuracy. This information can be used to retrain and improve the model, ensuring it can effectively distinguish vehicles and parking spaces in diverse lighting and environmental conditions.

- **Algorithm Optimization**

The algorithms for empty space calculation, labeling, and car size estimation can also be further optimized based on real-world data. This ongoing improvement process ensures the system maintains a high level of accuracy and effectiveness.

- **User Feedback**

Feedback from users is also integrated into the improvement process. This feedback helps identify areas where the system can be enhanced, ensuring that Rakeny continues to meet the needs of its users effectively.

By integrating these components and continuously refining the system based on real-world data, Rakeny delivers a robust and user-friendly parking management solution that addresses the challenges of urban parking in a scalable and efficient manner.

4.2 AI Model

4.2.1 Overview

The AI model designed for parking space detection and management leverages state-of-the-art deep learning techniques to efficiently and accurately identify available parking spaces in real-time. The primary objective of this model is to assist users in finding vacant parking spots, thus improving parking efficiency and reducing the time spent searching for parking. The model integrates computer vision, machine learning, and advanced image processing to achieve its goals.

4.2.2 Architecture and Components

The AI model comprises several key components, each playing a crucial role in the detection and management of parking spaces. These components include the convolutional neural network (CNN) for object detection, the post-processing algorithms for parking space identification, and the integration mechanism for real-time deployment.

4.2.3 Convolutional Neural Network (CNN) for Object Detection

At the heart of the AI model is a convolutional neural network (CNN), specifically the YOLOv8 (You Only Look Once, version 8) architecture.

YOLOv8 was chosen for its efficiency and accuracy in real-time object detection. It is designed to detect multiple objects within an image and classify them accordingly. The primary function of the CNN in this context is to identify and classify vehicles within the camera's field of view.

- **Input Layer**

The input layer of the CNN takes in frames from the camera feed. Each frame is an image represented as a matrix of pixel values.

- **Convolutional Layers**

These layers apply a series of convolution operations using various filters to extract features from the input images. Convolutional layers are followed by activation functions, typically ReLU (Rectified Linear Unit), which introduce non-linearity into the model.

- **Pooling Layers**

Pooling layers reduce the dimensionality of the feature maps, retaining the most important information while reducing computational load.

- **Fully Connected Layers**

The final layers of the CNN are fully connected layers that combine the extracted features to classify objects within the image. In this case, the CNN is trained to classify vehicles.

- **Output Layer**

The output layer provides bounding boxes around detected vehicles along with confidence scores and class labels.

4.2.4 Post-Processing for Parking Space Identification

Once vehicles are detected and classified by CNN, the model proceeds to identify potential parking spaces through a series of post-processing steps. These steps involve analyzing the spatial relationships between detected vehicles and the predefined parking lot layout.

- **Bounding Box Analysis**

The bounding boxes around detected vehicles are analyzed to determine their positions and dimensions within the camera frame.

- **Spatial Filtering**

The model applies spatial filtering to identify gaps between vehicles that are sufficiently large to be considered parking spaces. This involves calculating the distances between adjacent bounding boxes and comparing them to a predefined threshold that represents the minimum width of a parking space.

- **Occupancy Status Determination**

The model determines the occupancy status of each identified parking space by comparing the detected vehicle positions with the predefined parking space coordinates. If a bounding box overlaps with a parking space, it is marked as occupied; otherwise, it is marked as vacant.

4.2.5 Car Fitting Analysis

In addition to identifying vacant parking spaces, the model also assesses whether a detected parking space can accommodate a specific vehicle. This is achieved through the following process:

- **Vehicle Dimensions Input**

The model takes the dimensions of the user's vehicle as input. These dimensions include the vehicle's width, length, and height.

- **Space Measurement**

For each identified vacant parking space, the model calculates the available width.

- **Fit Assessment**

The model compares the dimensions of the vacant space with those of the vehicle. If the space's dimensions are greater than or equal to the vehicle's dimensions, the space is marked as suitable; otherwise, it is marked as unsuitable.

- **Visualization**

Suitable parking spaces are highlighted in a distinct color on the visual feed, providing users with clear and immediate information about where they can park their vehicle.

4.2.6 Real-Time Deployment and Integration

The AI model is designed for real-time deployment, making it capable of processing live camera feeds and providing immediate feedback on parking space availability. Integration with the mobile application and camera infrastructure is crucial for seamless functionality.

- **Camera Integration**

The model interfaces with both local and remote cameras, capturing live feeds to be processed. Camera calibration ensures that the feeds are correctly aligned with the physical parking lot layout.

- **Mobile Application Interface**

The model integrates with a mobile application that allows users to input their current location or the location which the user desire to find parking spaces at and receive real-time updates on available parking spaces nearby. The application utilizes the model's outputs to provide precise coordinates and navigation assistance to the users.

- **Scalability and Flexibility**

The AI model is designed to be scalable, capable of handling multiple camera feeds simultaneously. It is flexible enough to be adapted to various parking lot configurations and sizes, making it suitable for diverse deployment scenarios.

4.2.7 Location Calculation and Feedback

A critical feature of the AI model is its ability to calculate the exact geographic coordinates of vacant parking spaces and send this information back to the user.

- **Camera Position Calibration**

Each camera's position is calibrated and mapped to geographic coordinates. This involves determining the precise latitude and longitude of each camera and understanding the field of view and coverage area.

- **Coordinate Transformation**

Once a vacant parking space is identified within the camera feed, its position in the image is transformed into real-world coordinates using the camera's calibration data. This transformation accounts for the camera's angle, height, and other relevant factors to ensure accurate mapping.

- **User Feedback**

The calculated coordinates of the vacant parking spaces are then transmitted to the mobile application. Users receive these coordinates along with the distance and direction from their current location, enabling precise navigation to the available parking spot.

4.2.8 Conclusion

The AI model for parking space detection and management represents a significant advancement in smart parking solutions. By integrating cutting-edge deep learning techniques with real-time processing capabilities, the model provides an effective and efficient means of managing parking spaces.

Its ability to accurately detect vehicles, identify vacant spaces, assess fit for specific vehicles, and calculate precise location coordinates makes it a valuable tool in addressing the challenges of urban parking.

4.3 Admin Interface

4.3.1 Overview of the Admin Interface

The admin interface for Rakeny is meticulously designed to provide a robust and user-friendly platform for parking facility owners. The development of this interface necessitated the selection of tools, languages, and platforms that ensure the code is efficient, scalable, and maintainable.

Key components of the development environment include Python, chosen for its versatility and simplicity, and Streamlit, selected for its capability to create intuitive web applications. Additionally, various supporting libraries and frameworks were employed to enhance functionality and user experience.

This combination of technologies results in a powerful interface that meets the complex needs of parking facility management, ensuring users can easily navigate and utilize its features for optimal performance.

The design and functionality of the admin interface aim to address the specific challenges faced by parking facility managers. From monitoring parking activity in real-time to managing dynamic pricing strategies, the interface provides a comprehensive suite of tools that streamline operations and improve decision-making processes.

The subsequent sections will detail the key features and benefits of the admin interface, highlighting how each component contributes to an efficient and effective parking management system.

4.3.2 Key Features

1. Centralized Monitoring

- **Real-Time Overview:** The interface provides a real-time overview of parking activity, displaying an aerial view of parking areas with real-time updates on available and occupied spots. This feature is crucial for facility managers to maintain situational awareness and respond promptly to changing conditions.

2. Usage Analysis

- **Trend Analysis:** Tracks and analyzes parking trends over time, offering insights into peak usage times, average parking durations, and turnover rates. This data is invaluable for making informed decisions about capacity planning and operational adjustments.

3. Rate Management

- **Dynamic Pricing:** Allows facility owners to set and adjust parking rates dynamically. Supports different pricing strategies such as hourly rates, daily rates, and special event pricing. This flexibility ensures that pricing can be optimized based on demand and other factors.

4. Reporting and Insights

- **Detailed Reports:** Generates detailed reports on parking usage, revenue, and occupancy rates. These reports provide a comprehensive overview of the facility's performance and are essential for strategic planning.
- **Predictive Analytics:** Provides predictive analytics to forecast future parking demand and optimize resource allocation. This forward-looking feature helps in anticipating changes and preparing accordingly.

5. User Management

- **Account Management:** Facilitates the management of user accounts and access permissions. This ensures that only authorized personnel have access to specific features and data.
- **Secure Access:** Ensures secure access to the admin interface with role-based access controls. This is critical for maintaining data security and operational integrity.

Table 2: Admin Interface User Roles and Permissions

User Role	Permissions
Super Admin	• Add new cameras to the system. • Perform camera calibration using a checkerboard pattern. • Oversee dynamic data processing for cameras every 30 seconds. • Access and manage all administrative functions of the system.
Sub Admin	• Manage parking spaces through the control room page. • Visualize parking spaces for each camera in the system, including vehicle and free space detection results. • Generate reports about the parking area for effective management. • Access control room functionalities but with restricted access compared to Super Admins.

6. Integration with Guidance System

- **Synchronization:** Synchronizes with the guidance system to ensure drivers receive accurate and up-to-date parking information. This integration enhances the overall parking experience by reducing the time spent searching for available spots.
- **Improved Experience:** Enhances the overall parking experience by minimizing the time spent searching for available spots. Accurate and timely information helps drivers make informed decisions quickly.

7. Alerts and Notifications

- **Critical Alerts:** Sends alerts for critical events such as over-occupancy, equipment malfunctions, or security breaches. Timely alerts help in mitigating potential issues before they escalate.
- **Customization:** Allows customization of notification preferences based on user roles and responsibilities. This ensures that relevant personnel are notified about issues pertinent to their roles.

4.3.3 Benefits

Operational Efficiency

The admin interface streamlines the management of parking facilities, reducing manual effort and increasing operational efficiency. By automating various processes, it allows facility owners to focus on more strategic tasks.

The reduction in manual workload also reduces the risk of human error, leading to more reliable and consistent operations.

Data-Driven Decision Making

The interface empowers parking facility owners with actionable insights to make informed decisions regarding pricing, resource allocation, and future expansions. The data collected and analyzed enables a deeper understanding of usage patterns and operational bottlenecks.

This data-driven approach ensures that decisions are based on accurate and relevant information, leading to more effective management.

Enhanced Customer Experience

By ensuring the availability of parking spots and reducing search time, the interface significantly improves the overall parking experience for drivers. Integration with guidance systems and real-time updates help drivers quickly find available spots, enhancing convenience and satisfaction.

A better parking experience can also lead to increased customer loyalty and repeat business, further benefiting the facility's performance.

4.4 Mobile Application

Mobile application "Rakeny" is designed to provide an intuitive and efficient platform for users to locate and navigate to available parking spots. Utilizing cutting-edge technologies in mobile development, machine learning, artificial intelligence, and computer vision, Rakeny aims to revolutionize the parking experience by offering real-time, accurate, and user-friendly services.

This section elaborates on the design principles and key functionalities of the application.

4.4.1 Design Features

4.4.1.1 User-Centric Design

The design of Rakeny prioritizes user experience, ensuring the application is easy to navigate and use. Key design principles include simplicity, consistency, and responsiveness. The user interface (UI) is crafted to minimize the number of steps required to perform core functions, reducing user effort and improving satisfaction.

4.4.1.2 Tools and Technologies

Rakeny is developed using a combination of powerful tools and technologies, ensuring a robust and scalable solution for users seeking efficient parking management. The integration of these technologies allows for seamless operation, high performance, and an intuitive user experience.

Flutter

Overview:

- Flutter provides a comprehensive framework for creating cross-platform applications, enabling developers to build natively compiled applications for mobile, web, and desktop from a single codebase.

Key Features:

- **Pre-designed Widgets:** A rich set of customizable widgets allows for the creation of visually appealing and highly interactive UIs.
- **High-Performance UI Components:** Ensures smooth and responsive applications.
- **Optimal Performance:** The use of Flutter's main SDK guarantees efficient performance and quick rendering of the application interface.
- **Seamless Integration:** Flutter's architecture supports the easy integration of various features, enhancing the overall functionality of the application.

Flutter Flow

Overview:

- Flutter Flow is a visual development platform that accelerates the development process by providing an intuitive interface for designing and building Flutter applications.

Key Features:

- **Rapid Prototyping:** Enables quick creation and testing of application prototypes.
- **Design Integration:** Facilitates seamless integration of design elements, allowing developers to create aesthetically pleasing and functional UIs without extensive coding.
- **User-Friendly:** Simplifies the development process, making it accessible even to those with minimal coding experience.

Firebase

Overview:

- Firebase offers a comprehensive suite of backend services essential for modern application development, including real-time data synchronization, user authentication, and database management.

Key Features:

- **Real-Time Data Synchronization:** Ensures that data is instantly updated across all devices, providing a consistent user experience.
- **Authentication:** Provides secure and straightforward methods for user authentication, including email/password, social media logins, and more.
- **Database Management:** Utilizes Firestore, a flexible, scalable database for mobile, web, and server development, ensuring efficient data handling and storage.

Google Cloud Platform

Overview:

- Google Cloud Platform (GCP) provides a suite of cloud computing services that runs on the same infrastructure that Google uses internally for its end-user products.

Key Features:

- **Scalable Cloud Services:** Supports the application's backend processing and storage needs, ensuring scalability and reliability.
- **Infrastructure:** Offers robust infrastructure solutions, including virtual machines, storage options, and networking capabilities.
- **AI and ML Tools:** Provides access to advanced AI and machine learning tools that can be integrated to enhance the application's functionality.
- **Google Maps API:** Integrates essential APIs for Google Maps, enabling location services, real-time navigation, and geospatial data processing within the application.

MongoDB

Overview:

- MongoDB is a powerful, flexible NoSQL database that stores data in JSON-like documents, making it easy to store, query, and retrieve large amounts of data.

Key Features:

- **Flexible Schema:** Allows for the storage of diverse types of data without the need for a predefined schema.
- **Scalability:** Supports horizontal scaling through sharding, making it ideal for handling large volumes of data.
- **High Performance:** Optimized for read and write operations, ensuring fast data access and manipulation.
- **Rich Query Language:** Provides a powerful query language for efficient data retrieval and manipulation.

4.4.1.3. Key UI Components

- **Onboarding Page:** The Onboarding Page of the Rakeny mobile application serves as the initial introduction for new users, providing a guided experience through the app's features and setup process.
- **Authentication Page:** This page allows users to log in or sign up. It features simple input fields for username and password, along with options for password recovery and social media login.
- **Home Page:** Users can choose to find a parking spot immediately or search for a location on Google Maps. The page is designed with prominent buttons and a user-friendly map interface to facilitate easy navigation.
- **Navigation Page:** Integrated with Google Maps, this page provides turn-by-turn directions to the parking spot identified by the AI model based on the user's specified location. The map updates in real-time to ensure accurate guidance.
- **Profile Page:** This page displays the user's information and offers an option to edit data. The layout is clean and organized, making it easy for users to view and update their personal details.
- **History Page:** A comprehensive list of all the parking spots the user has previously visited is displayed here. This page provides a chronological history, allowing users to review their past parking locations.
- **Settings Page:** This page includes options for logging out, finding help, following Rakeny on social media, and learning about the application. The settings are organized into categories for easy access.
- **Edit Data Page:** Users can update their personal information on this page. Input fields are clearly labeled, and validation ensures that data is entered correctly.

4.4.2 Functionality

Rakeny's mobile application offers a range of innovative features designed to streamline the parking experience for users. Below is a detailed overview of the key functionalities, supported by the tools and technologies used in the development process.

4.4.2.1. Car Dimensions Recognition

- **Overview:** This feature enables Rakeny to recognize car dimensions and match them with suitable parking spots. It ensures users are directed to spots that can accommodate their vehicles, improving convenience and reducing the likelihood of parking issues.
- **Machine Learning Model:** Trained on a diverse dataset of car types and their dimensions.
- **Technology Stack:**
 - **Flutter Flow:** Provides a rich set of pre-designed widgets for the user interface, displaying car dimensions and matching parking spots.
 - **MongoDB:** Stores the dataset of car types and dimensions, ensuring flexible and efficient data retrieval.

4.4.2.2. Navigation System

- **Overview:** Rakeny's navigation system uses GPS and real-time data to guide users to the nearest available parking spot, whether indoors or outdoors. The system provides precise directions, making the journey to the parking spot seamless.
- **Real-Time Data:** Updates users on available parking spots based on their current location.
- **Technology Stack:**
 - **Flutter Flow:** Delivers a smooth and responsive navigation interface. Enables rapid prototyping and design integration for the navigation system.
 - **Google Maps API:** Provides location services, real-time navigation, and geospatial data processing.

4.4.2.3. Spot Saver

- **Overview:** The spot saver feature allows users to save the location of their parked car using geolocation tagging. When users need to find their car, the app provides directions back to the saved location, eliminating the common problem of forgetting where one parked.
- **Geolocation Tagging:** Utilizes GPS to tag and save the car's location.
- **Technology Stack:**
 - **Flutter Flow:** Ensures a user-friendly interface for saving and retrieving parking spot locations.

4.4.2.4. Automated Parking Payment (Future Update)

- **Overview:** This planned feature will use computer vision to capture and recognize license plate information. It will track the duration of parking and automatically process payments based on the time parked, streamlining the parking experience and reducing the need for manual payment.
- **Computer Vision:** Recognizes license plates and tracks parking duration.
- **Technology Stack:**
 - **Flutter Flow:** Provides a high-performance interface for displaying payment information.
 - **MongoDB:** Stores license plate data and payment history, ensuring efficient data management and retrieval.

4.5 Integration

How the components are integrated to form the complete system.

4.5.1 Integration Overview

The successful integration of various components is essential for the seamless operation of Rakeny, the smart parking system platform. This section elaborates on how the MongoDB database, the admin interface, and the mobile user interface are integrated to form a cohesive and efficient system.

4.5.2 MongoDB Database

MongoDB, a NoSQL database, was chosen for its flexibility, scalability, and ease of use, making it ideal for managing the complex and dynamic data requirements of Rakeny. MongoDB's document-oriented data model allows for the storage of diverse data types, such as real-time parking availability, user profiles, and historical parking data.

Key Features of MongoDB

1. **Scalability:** MongoDB's horizontal scaling capabilities ensure that the database can handle increasing loads and data volumes, crucial for an urban parking system expected to serve thousands of users simultaneously.
2. **Flexibility:** The document-oriented model allows for the storage of complex data structures in a single collection, facilitating the management of varied data such as user preferences, parking spot statuses, and transaction records.
3. **Real-Time Data Processing:** MongoDB supports real-time data processing, which is critical for updating parking spot availability and user interactions instantaneously.
4. **Robust Querying Capabilities:** MongoDB's powerful querying and indexing features enable efficient retrieval of data, ensuring quick responses to user queries and administrative requests.

4.5.3 Integration with the Admin Interface

The admin interface, built using Streamlit, connects directly to the MongoDB database to provide parking facility managers with real-time insights and control over their operations. The integration ensures that the admin interface can perform the following functions:

1. **Real-Time Monitoring:** The admin interface retrieves live data from MongoDB to display the status of all parking spots, including availability and occupancy rates. This is visualized through an intuitive dashboard that offers an aerial view of parking facilities.
2. **User and Access Management:** The admin interface manages user accounts and roles, ensuring secure access to the system. Data related to user roles and permissions are stored and managed within MongoDB, providing a centralized and secure method of access control.

4.5.4 Integration with the Mobile User Interface

The mobile user interface is designed to offer drivers a seamless experience in locating and reserving parking spots. Integration with the MongoDB database allows the mobile app to:

1. **Real-Time Parking Availability:** The app fetches real-time data on available parking spots from MongoDB, guiding drivers to the nearest available spaces. This reduces the time spent searching for parking and enhances the user experience.
2. **Parking History and Notifications:** User-specific data, such as parking history and reminders of parked locations, are stored in MongoDB. This data is accessible via the mobile app, providing users with personalized and relevant information.

4.5.6 Data Flow and Synchronization

Data flow between the components of the Rakeny system is managed through MongoDB Atlas HTTP endpoints (Data API). These endpoints handle requests for data retrieval and updates, ensuring efficient and secure communication between the admin interface, mobile user interface, and the MongoDB database.

1. RESTful API Integration

The MongoDB Atlas Data API provides RESTful HTTP endpoints that facilitate data operations. These endpoints are used by both the admin and user interfaces to perform CRUD (Create, Read, Update, Delete) operations on the MongoDB database. This setup ensures that data is accessed and manipulated in a consistent and reliable manner.

2. Data Retrieval and Updates

The admin interface uses the Data API to fetch real-time data on parking spot availability, user management, and analytics. Similarly, the mobile user interface utilizes these endpoints to access parking availability and retrieve user-specific data such as user authentication, parking history and notifications. The use of HTTP endpoints ensures that all data transactions are secure and authenticated, providing a robust framework for data operations.

4.5.7 Conclusion

The integration of MongoDB with the admin interface and mobile user interface forms the backbone of the Rakeny system, ensuring efficient data management, real-time updates, and seamless user experiences.

By leveraging MongoDB Atlas HTTP endpoints, Rakeny ensures that data flow is managed efficiently and securely, providing a seamless and integrated experience for both parking facility managers and drivers. This architecture enhances the system's scalability and reliability, positioning Rakeny as a cutting-edge solution for modern urban parking challenges.

Chapter 5: Implementation

5.1 Development Environment

The development of Rakeny leveraged a diverse set of tools, programming languages, and cloud platforms to create a comprehensive and integrated parking management solution. This section details the specific technologies employed for each component of the system:

5.1.1. AI Model Development

- **YOLOv8 Deep Learning Model**

The core of Rakeny's object detection capabilities is the YOLOv8 deep learning model. This pre-trained model, known for its speed and accuracy, was chosen for its suitability in real-time vehicle detection tasks.

- **Camera Calibration with OpenCV**

For optimal performance in real-world scenarios, camera calibration is often essential. OpenCV, a popular open-source library for computer vision tasks in Python, likely provided the tools necessary for this process. Camera calibration helps to correct for distortions inherent in camera lenses, ensuring accurate object detection and spatial measurements within the captured video feed.

- **Geospatial Calculations**

Transforming the raw object detection data from the AI model into actionable information for the user interface involves precise geospatial calculations. The algorithm calculates the Field of View (FOV) per pixel using the camera matrix and image dimensions. By converting pixel coordinates to GPS coordinates based on the camera's latitude, longitude, and altitude, the system can accurately locate empty parking spaces in real-world coordinates. This transformation is crucial for guiding drivers to available spots.

5.1.2. Admin Interface Development

- **Streamlit Platform**

Rakeny's web-based admin interface was developed using Streamlit, a Python framework specifically designed for creating data apps. Streamlit simplifies the process of building interactive dashboards and visualizations, allowing developers to focus on the core functionalities of the admin interface.

- **Python, CSS, and Bootstrap**

The front-end development of the admin interface likely involved a combination of CSS (Cascading Style Sheets) for styling the visual elements, Python for core functionality and data manipulation, and Bootstrap, a popular CSS framework, for creating responsive and user-friendly layouts.

These tools allowed for the creation of a visually appealing and intuitive admin interface that empowers authorized personnel to monitor and manage the parking system.

5.1.3. User Interface Development

- **Flutter and Dart**

The user-friendly mobile application for Rakeny was developed using Flutter, a popular framework for building cross-platform applications. Flutter utilizes Dart, a versatile object-oriented programming language, to create a single codebase that can be deployed on both Android and iOS devices.

- **Flutterflow Platform**

While the core functionalities of the mobile application were built using Flutter and Dart, Flutterflow development platforms was used to accelerate the process. This platform offers pre-built UI components and functionalities that streamlined the development process, allowing us to focus on the unique features of the Rakeny application.

- **Firebase and Google Cloud Platform (GCP) for APIs**

Firebase, a suite of tools and services offered by Google, played a crucial role in the backend development of the mobile application.

Firebase offers functionalities for user authentication, real-time database management, and cloud functions. Additionally, Google Cloud Platform (GCP) was utilized for hosting backend APIs (Application Programming Interfaces) that facilitate communication between the mobile application and the core Rakeny system.

5.1.4. System Integration

- **MongoDB HTTP Endpoints API**

MongoDB's HTTP Endpoints API played a critical role in data storage and retrieval. This API facilitates secure and efficient interaction with the MongoDB database, enabling real-time updates and queries essential for the dynamic nature of Rakeny. It ensures seamless data communication between the detection system and the user interfaces, maintaining up-to-date parking availability information.

5.2 AI Model Implementation

This section provides an in-depth technical overview of the AI model developed for parking space detection and management, detailing the algorithms, processes, and technologies used.

5.2.1 Overview of the YOLOv8 Model

YOLOv8 (You Only Look Once, version 8) is a state-of-the-art object detection model known for its efficiency and accuracy. Unlike traditional object detection models that process images in multiple stages, YOLOv8 processes an image in a single pass.

This end-to-end approach allows the model to predict bounding boxes and class probabilities directly from the image, making it highly suitable for real-time applications such as parking space detection.

5.2.2 Architecture:

- **Convolutional Layers**

These layers extract spatial features from the input images using convolution operations. Convolutional layers apply a series of filters to the input image, highlighting features such as edges, textures, and patterns that are essential for object detection.

The layers are designed to capture hierarchical features, starting with low-level features in the initial layers and progressing to high-level features in the deeper layers.

- **Pooling Layers**

Pooling layers, often max-pooling, reduce the spatial dimensions of the feature maps. This down-sampling operation helps in reducing the computational complexity and the number of parameters while retaining the most important features.

Pooling also provides a form of spatial invariance, making the model more robust to variations in object position.

- **Fully Connected Layers**

These layers combine the extracted features and perform the final classification and regression tasks. The fully connected layers take the high-level features from the convolutional layers and predict the class probabilities and bounding box coordinates. This dense connectivity allows the model to integrate information from all parts of the image.

- **Output Layer**

The output layer of YOLOv8 generates the final predictions, including class labels and bounding box coordinates for each detected object in the image.

The predictions are organized into a grid structure, with each cell responsible for predicting a fixed number of bounding boxes.

5.2.3 Detecting Vehicles

Vehicle detection is the fundamental step in the management of parking spaces. The YOLOv8 model is employed to identify and locate vehicles in the camera feeds.

Process

- **Preprocessing**

The input images are resized to a standard size and normalized to enhance model performance. This involves scaling the pixel values to a range of 0 to 1, which helps in speeding up the training and inference processes.

Additionally, data augmentation techniques such as flipping, rotation, and scaling may be applied to improve the model's robustness to variations in the input data.

- **Detection**

The preprocessed images are fed into the YOLOv8 model. The model uses its convolutional neural network (CNN) to analyze the images and generate bounding boxes around detected vehicles.

Each bounding box is associated with a confidence score and a class probability, indicating the likelihood of the object being a vehicle. The model outputs multiple bounding boxes per grid cell, which are then filtered based on confidence scores.

- **Post-processing**

To refine the model's predictions, Non-Maximum Suppression (NMS) is applied. NMS eliminates redundant bounding boxes by retaining only the most confident predictions, thus ensuring that each vehicle is detected only once.

The NMS algorithm sorts the bounding boxes by their confidence scores and iteratively removes boxes that have a high overlap with previously selected boxes.

5.2.4 Parking Space Detection

Identifying empty parking spaces involves analyzing the areas between detected vehicles. This step is crucial for determining where new vehicles can be parked.

Process

- **Sorting Bounding Boxes**

The bounding boxes of detected vehicles are sorted based on their horizontal (x-axis) coordinates. This sorting is necessary to evaluate the gaps between consecutive vehicles accurately. Sorting helps in systematically analyzing the spatial distribution of vehicles and identifying potential parking spaces.

- **Calculating Gaps**

The horizontal gaps between consecutive bounding boxes are measured. The distance between the end of one bounding box and the start of the next bounding box is computed. These

gaps represent potential parking spaces. The gap calculation considers both the width and height dimensions to ensure that the identified spaces are viable for parking.

- **Threshold Check**

Gaps that are wider than a predefined threshold are identified as potential parking spaces. This threshold is determined based on the average width of a standard parking space. By setting an appropriate threshold, the model can effectively distinguish between viable parking spaces and smaller gaps that cannot accommodate vehicles.

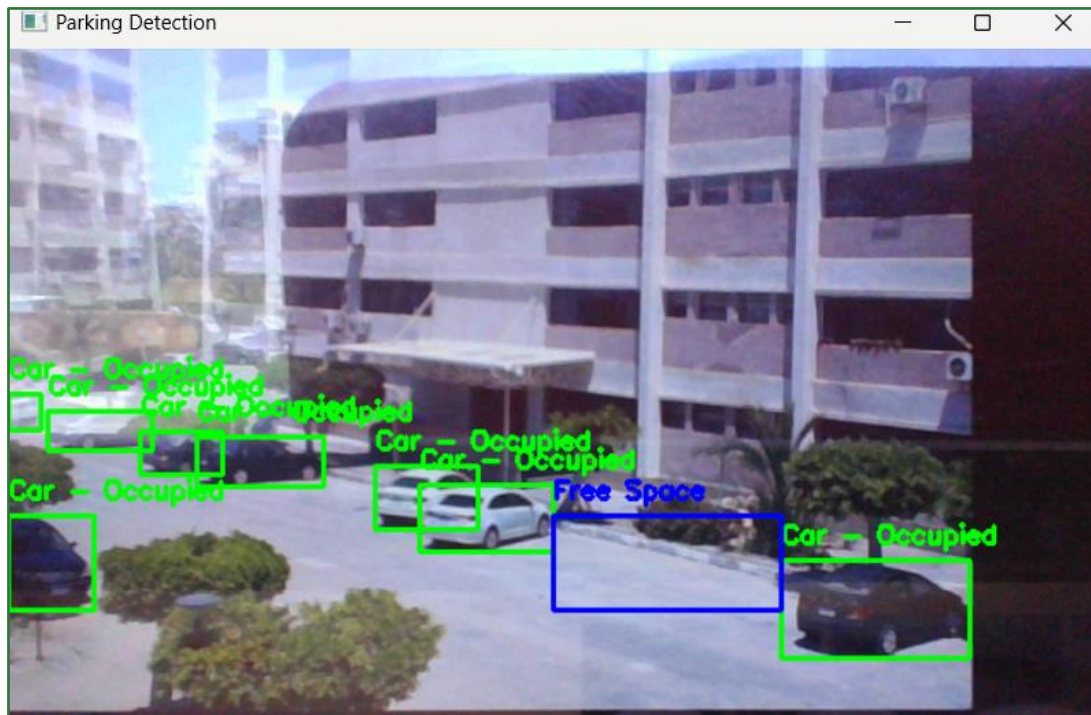


Figure 4: Vehicles and Parking Space Detection

5.2.5 Camera Calibration

Accurate detection of parking spaces requires precise calibration of the camera to map pixel coordinates to real-world coordinates.

Process

- **Camera Calibration**

Camera calibration is a critical process in computer vision and photogrammetry, essential for obtaining precise measurements and accurate representations of the captured scene. The calibration procedure involves estimating the intrinsic and extrinsic parameters of the camera, encapsulated in the camera matrix, distortion coefficients, rotation vectors, and translation vectors.

The camera matrix (or intrinsic parameters) defines the focal length and the optical center of the camera, while the distortion coefficients account for lens distortions such as radial and tangential distortions.

Undistortion is the process of correcting these lens distortions, ensuring that straight lines in the real world appear straight in the image, thus restoring the geometric integrity of the scene.

This correction is vital for applications like image stitching, 3D reconstruction, and augmented reality, where accurate geometry is paramount. The undistortion process uses the intrinsic parameters and distortion coefficients obtained from the calibration to rectify the image.

Reprojection, on the other hand, involves mapping 3D points in the world to 2D points in the image plane using the intrinsic parameters along with the rotation and translation vectors (extrinsic parameters).

This process is crucial for verifying the accuracy of the calibration, overlaying 3D models onto images in augmented reality, and for various other applications that require understanding the spatial relationship between 3D points and their 2D image counterparts.

In the context of this study, the calibration process was visually validated by detecting and marking the corners of a checkerboard pattern in the calibration images.

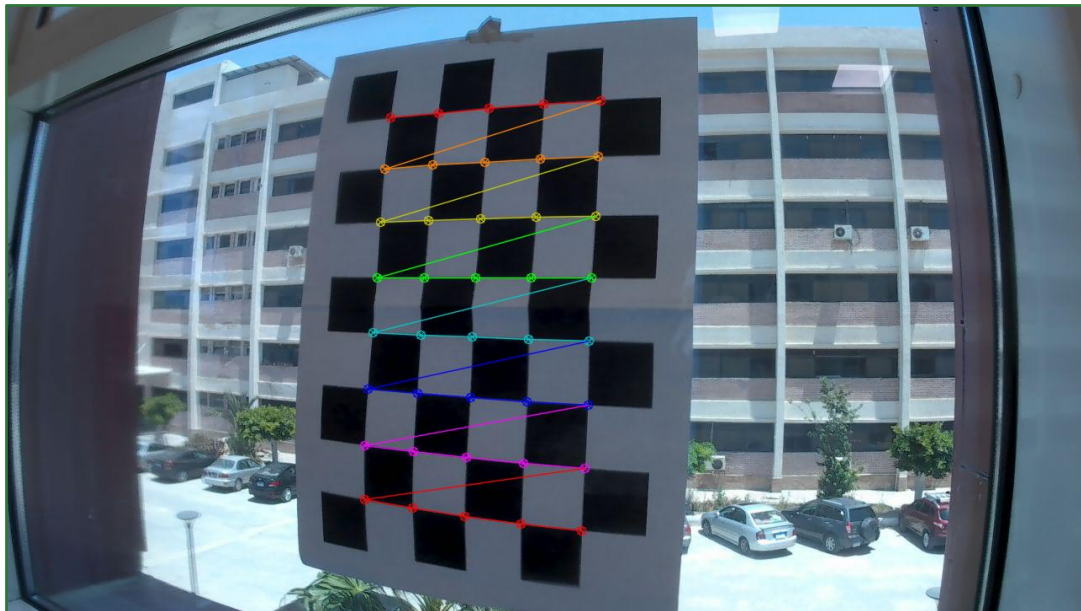


Figure 5: Camera Calibration Process

Figure 5 shows an image with the detected corners highlighted, demonstrating the accuracy of the corner detection algorithm. This step is crucial for ensuring that the collected data is reliable and that the subsequent calibration and undistortion steps will produce accurate and useful results.

The precise detection of these corners underpins the accuracy of the entire calibration process, providing a foundation for high-fidelity computer vision applications.

5.2.6 Car Fit Logic

To determine if a detected space can accommodate a vehicle, the model compares the dimensions of the detected space with the dimensions of a standard vehicle.

Process

- **Bounding Box Dimensions**

The width and height of the detected parking space are measured. These dimensions are extracted from the bounding box coordinates, representing the spatial extent of the identified space.

- **Vehicle Dimensions**

Predefined dimensions of standard vehicles are used as a reference. These dimensions are typically based on average measurements of common vehicle types, ensuring that the detection system can accommodate a wide range of vehicles.

- **Comparison**

The dimensions of the parking space are compared with the vehicle dimensions. If the space dimensions are greater than or equal to the vehicle dimensions, the space is deemed suitable for parking. This comparison involves simple arithmetic operations and logical checks to determine the suitability of each identified space.

5.2.7 Drawing Bounding Boxes

Bounding boxes are drawn around detected vehicles and parking spaces to visualize the detection results. This step is essential for providing feedback to users and for validating the detection process.

Process

- **Drawing Boxes**

For each detected vehicle and parking space, a rectangle is drawn around the object using the bounding box coordinates. The bounding boxes are visual indicators that highlight the detected objects in the camera feed.

- **Labeling Boxes**

Each bounding box is annotated with relevant information such as "Car 1" or "Free Space 1". This labeling helps in identifying and differentiating between vehicles and parking spaces.

Labels provide contextual information and improve the interpretability of the detection results.

- **Color Coding**

Different colors are used to indicate the status of the bounding boxes. Color coding enhances the visual distinction between different types of objects and their statuses.

5.2.8 Location Calculation with Camera Matrix and Field of View

The model calculates the geographic coordinates of detected parking spaces using a combination of camera matrix parameters and field of view calculations, enabling precise navigation.

Process

- **Coordinate Transformation**

Pixel coordinates of detected bounding boxes are converted to real-world coordinates using the camera matrix and image dimensions. This transformation involves applying geometric transformations to map the pixel values accurately. By determining the field of view (FOV) per pixel, the system translates the pixel coordinates to angles relative to the camera.

- **Field of View Calculations**

The FOV for the camera is calculated using the focal lengths (f_x , f_y) from the camera matrix and the dimensions of the captured image. The horizontal and vertical angles per pixel are determined by dividing the FOV by the image width and height, respectively. This step ensures that each pixel's position in the image is accurately mapped to a corresponding angle in the real world.

- **Geospatial Conversion**

Using the calculated angles and the camera's known latitude, longitude, and altitude, the system converts the angles to horizontal and vertical distances. These distances are then used to compute the geographic offsets (latitude and longitude) from the camera's position. The algorithm accounts for the Earth's curvature by considering the Earth's radius in the calculations.

By implementing these steps, the system can translate detected parking spaces' pixel coordinates to precise geographic coordinates, which are then stored in the MongoDB database. This process ensures that users receive accurate navigation information to available parking spots.

5.2.9 Integration with User and Admin Interfaces

The AI model is integrated with a Streamlit-based user interface to facilitate real-time interaction and management.

5.2.9.1 User Interface

- **Real-Time Camera Feed**

The user interface displays live feeds from the cameras, overlaying the detection results. This allows users to monitor parking spaces in real-time.

The interface provides a visual representation of the detection process and enables users to interact with the system.

- Interactive Controls**

Users can pause and resume camera feeds, adjust bounding boxes, and save configurations through the interface.

These controls enhance the usability and flexibility of the system. Interactive elements allow users to customize the detection process and adapt it to different environments and requirements.

5.2.9.2 Admin Interface

- Camera Management**

The admin interface enables administrators to define and save bounding boxes for specific areas in camera feeds.

This functionality is essential for setting up and configuring the detection system. Administrators can specify the regions of interest and ensure that the system focuses on relevant areas.

- Detection Configuration**

The admin interface provides options to configure detection parameters and thresholds. This allows administrators to fine-tune the detection process for optimal performance.

Configurable parameters enable the system to adapt to different conditions and improve detection accuracy.

5.2.10 Technologies and Libraries

The development of the AI model relies on a combination of advanced technologies and libraries.

Table 3: Technologies and Libraries Used in Artificial Intelligence Model

Technologies and Libraries Used in Artificial Intelligence Model	
YOLOv8	The primary object detection model used for detecting vehicles and parking spaces. YOLOv8 provides state-of-the-art performance in terms of speed and accuracy, making it suitable for real-time applications.
OpenCV	A powerful library for image processing and handling camera feeds. OpenCV offers a wide range of functions for image manipulation, feature extraction, and video capture, facilitating the development of the detection system.

5.3 Admin Interface Development

The development of the admin interface for Rakeny involved a careful selection of technologies and tools to ensure it meets the specific needs of parking facility management. The interface is designed to provide real-time updates, interactive features, and a user-friendly experience.

The following sections detail the key components, their rationale for use, and the implementation strategies employed.

5.3.1 Key Components

1. Streamlit Platform

Overview: Streamlit is an open-source app framework in Python that is particularly well-suited for building data-centric applications quickly and efficiently. It allows developers to create interactive web applications using simple Python scripts.

Rationale for Use: Streamlit was chosen for its ease of use, rapid development capabilities, and seamless integration with Python, which is the primary language used for the rest of the Rakeny system. Its ability to handle real-time data updates and interactive widgets made it ideal for creating a dynamic admin interface.

Implementation: The admin interface leverages Streamlit's features to provide real-time monitoring of parking spaces, display live camera feeds, and present interactive charts and graphs. Custom components and widgets were developed to meet the specific needs of parking facility management.

Streamlit's simple API allowed for rapid prototyping and iteration, ensuring that the interface remained responsive and easy to update as new requirements emerged.

2. Python

Overview: Python is a versatile, high-level programming language known for its readability and broad range of libraries and frameworks. It is widely used in data science, machine learning, and web development.

Rationale for Use: Python was selected due to its extensive support for machine learning and computer vision libraries such as OpenCV, TensorFlow, and scikit-learn, which are integral to the Rakeny system. Additionally, Python's simplicity and robust ecosystem allowed for rapid development and easy integration of various system components.

Implementation: The core logic for the admin interface, including data processing, analytics, and integration with the underlying machine learning (ML) and computer vision (CV) algorithms, was written in Python.

The use of Python ensured that the interface could seamlessly interact with other parts of the Rakeny system. Python scripts handled data ingestion, real-time updates, and communication with the database, ensuring that parking spot availability and other critical information were always up to date.

3. Bootstrap

Overview: Bootstrap is a popular front-end framework for developing responsive and mobile-first web applications. It provides a collection of pre-designed components, grid systems, and JavaScript plugins.

Rationale for Use: Bootstrap was utilized to ensure that the admin interface is visually appealing and user-friendly across different devices and screen sizes. Its components and layout utilities facilitated the creation of a professional and consistent user interface.

Implementation: Custom Bootstrap themes and components were integrated into the Streamlit application to enhance the interface's look and feel. This included the use of Bootstrap's grid system for layout management, buttons, modals, and navigation elements to improve the user experience.

The integration of Bootstrap ensured that the admin interface was not only functional but also aesthetically pleasing, providing an intuitive user experience.

4. JSON Files for Camera Mappings

Overview: JSON (JavaScript Object Notation) is a lightweight data interchange format that is easy for humans to read and write, and for machines to parse and generate.

Rationale for Use: JSON files were chosen to store camera mappings due to their simplicity and flexibility. They allow for easy configuration and updating of camera positions and parameters without the need for complex database management.

Implementation: Camera mappings, including details such as camera IDs, coordinates, and orientation, were stored in JSON files. These files were read by the Python backend to dynamically configure the monitoring system and ensure accurate real-time data visualization. Using JSON allowed for quick modifications and updates, which were crucial for maintaining accurate monitoring as the physical layout of parking facilities changed.

5.3.2 Integration and Workflow

The development environment for the Rakeny admin interface was designed to maximize productivity and maintainability. The workflow involved several key steps:

Initial Setup

Setting up the development environment with Streamlit, Python, and Bootstrap. This included installing necessary libraries and frameworks and configuring the development tools. The setup phase ensured that all components were properly integrated and ready for development.

Backend Development

Writing Python scripts to handle data processing, integrate machine learning models, and read camera mappings from JSON files. Ensuring the backend could handle real-time data updates and provide the necessary API endpoints for the front-end components. The backend development focused on creating a robust foundation for data handling and real-time updates.

Frontend Development

Utilizing Streamlit and Bootstrap to create interactive and responsive UI components. Customizing the look and feel of the interface using Bootstrap's theming capabilities. The frontend development aimed to create a user-friendly interface that facilitated easy interaction and data visualization.

Testing and Debugging

Rigorous testing of the interface to ensure functionality, performance, and usability. Debugging any issues that arose during the development process. Testing was an ongoing process, with both automated and manual testing methods employed to ensure the interface met all requirements and performed reliably under different conditions.

Deployment

Deploying the application in a suitable environment, ensuring that it could handle the load and provide real-time updates to users. The deployment phase included configuring servers, setting up continuous integration/continuous deployment (CI/CD) pipelines, and monitoring the application post-deployment to ensure it operated smoothly.

5.3.3 Database Integration

The admin interface is connected to the hosted database on MongoDB Atlas, which provides updated information about parking spot availability. This connection ensures that data displayed in the interface is always current, allowing for real-time monitoring and decision-making.

MongoDB Atlas

MongoDB Atlas is a fully managed cloud database service that handles the deployment, management, and scalability of MongoDB databases. It was chosen for its reliability, scalability, and ease of integration with Python applications.

Implementation

The admin interface interacts with MongoDB Atlas through the Python backend. Real-time updates on parking spot availability are fetched from the database and displayed in the interface. This integration ensures that facility managers have access to the most recent data, enabling them to make informed decisions quickly.

By detailing the development and integration of the admin interface, this section highlights the meticulous planning and execution that went into creating a robust and user-friendly system. The use of Streamlit, Python, Bootstrap, and JSON, along with the connection to MongoDB Atlas, underscores the thoughtful selection of technologies to meet the specific needs of parking facility management.

5.3.4 UI Components of Admin Interface

Authentication Page

- **Purpose:** Allows users to log in or sign up for the Rakeny application.
- **Features:**
 - **Log In:** User can enter as either admin or super-admin according to their credentials to access their account.

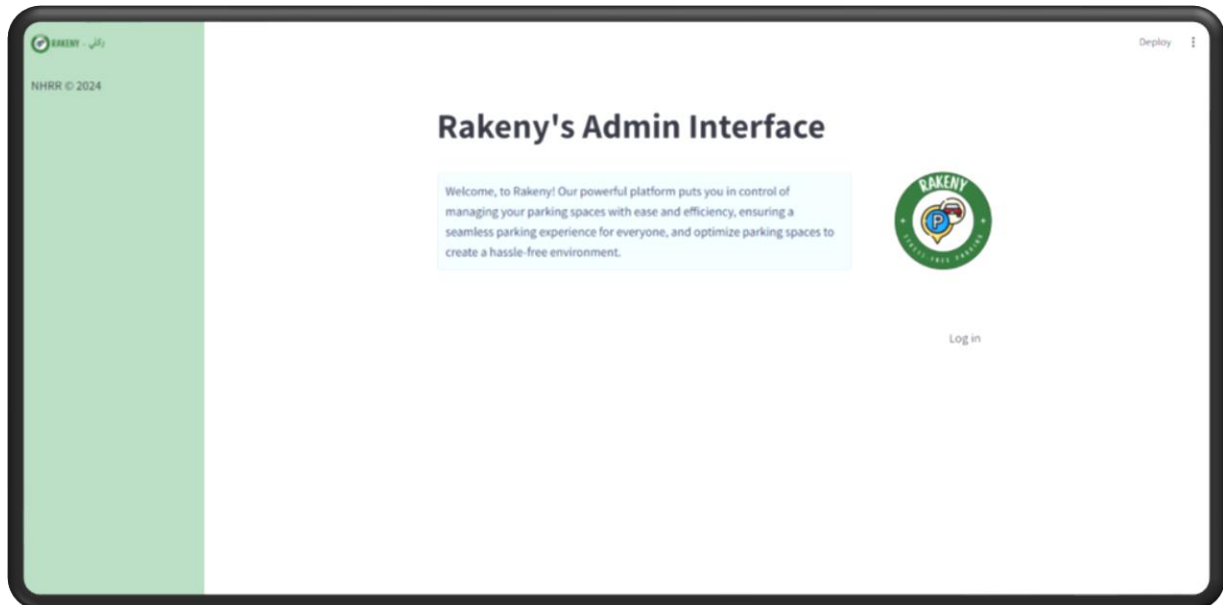


Figure 6: Admin Interface Login Page

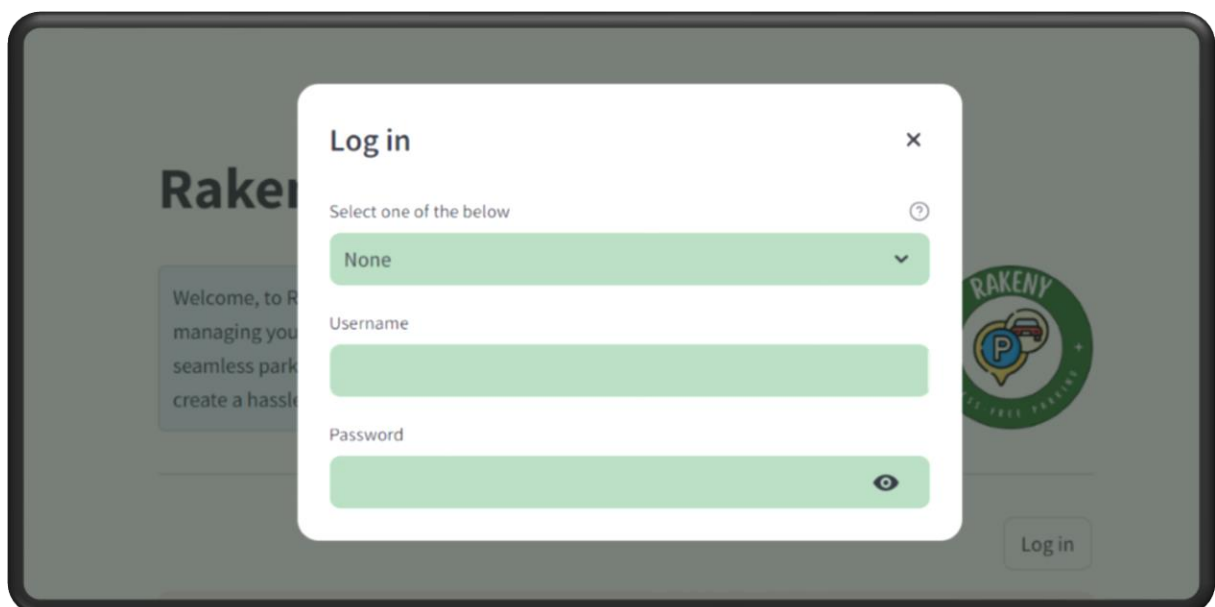


Figure 7: Admin Interface Login Page 2

Dashboard

- **Purpose:** Provides a comprehensive overview of the parking system's current status.
- **Features:**
 - **Real-Time Summary:** Displays the current number of occupied and available parking spaces.
 - **Visual Graphs:** Offers visual representations of parking usage trends over time.

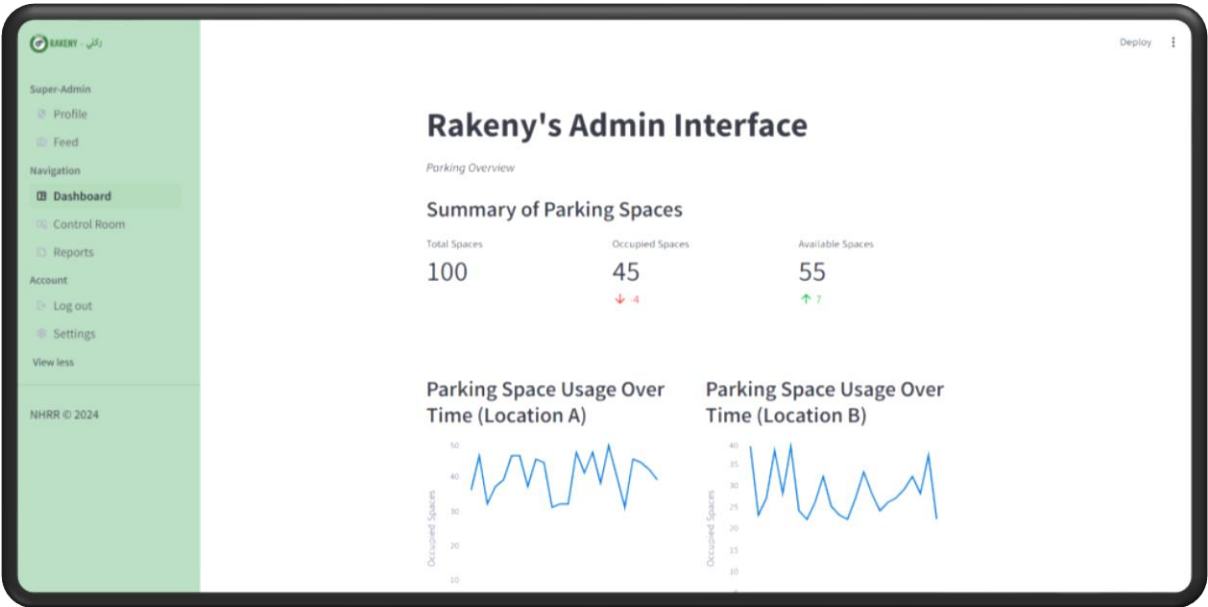


Figure 8: Admin Interface -Super Admin- Dashboard

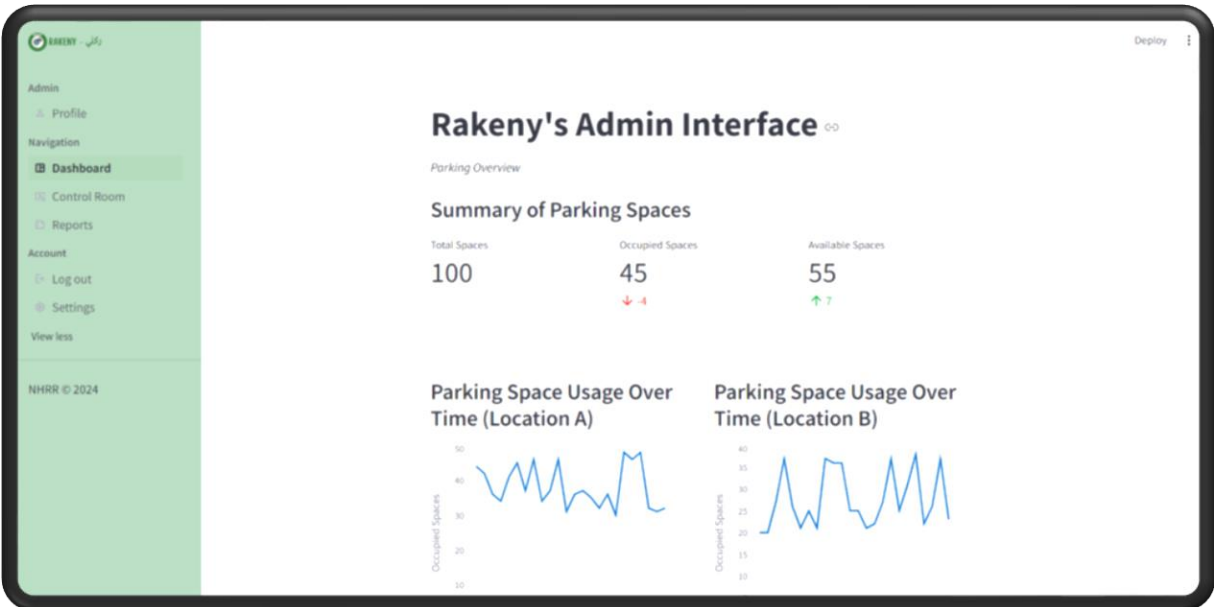


Figure 9: Admin Interface -Sub Admin- Dashboard

Feed

- **Purpose:** Enables super admins to add new cameras to the system, calibrate them and set desired bounding boxes over camera frames.
- **Features:**
 - **Add New Camera:** Add new cameras to the system.
 - **Review All Cameras & Test them:** Test camera feeds on the detection algorithm.
 - **Calibrate Cameras:** Perform calibration process on new cameras.

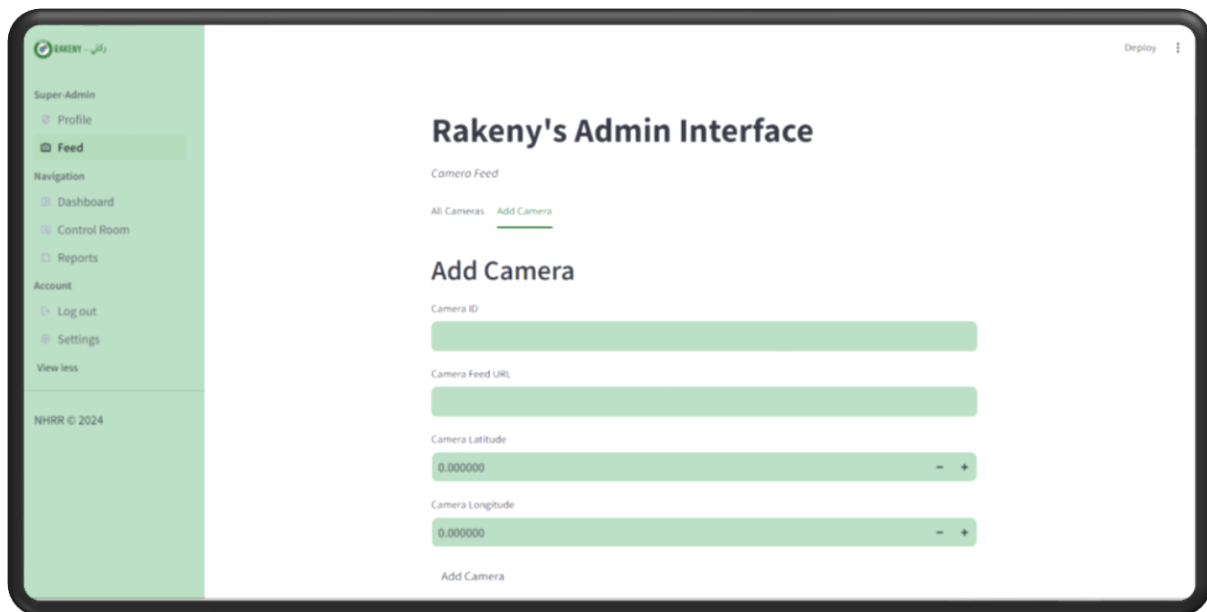


Figure 10: Admin Interface Feed (Add Camera Process)

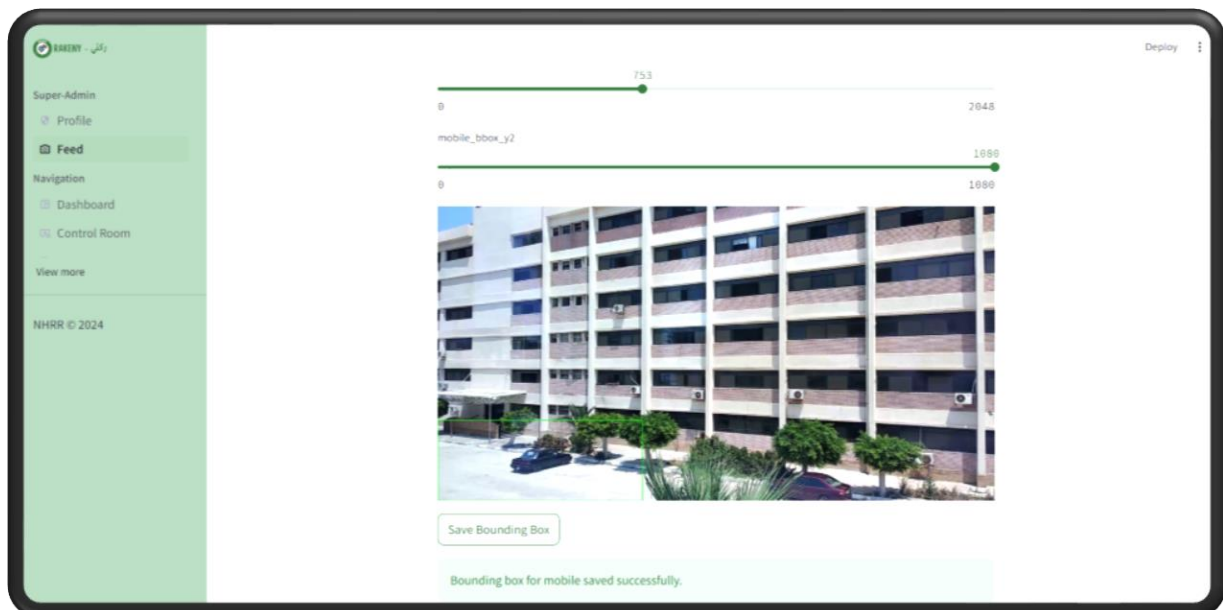


Figure 11: Admin Interface Feed (Add Bounding Box Process)

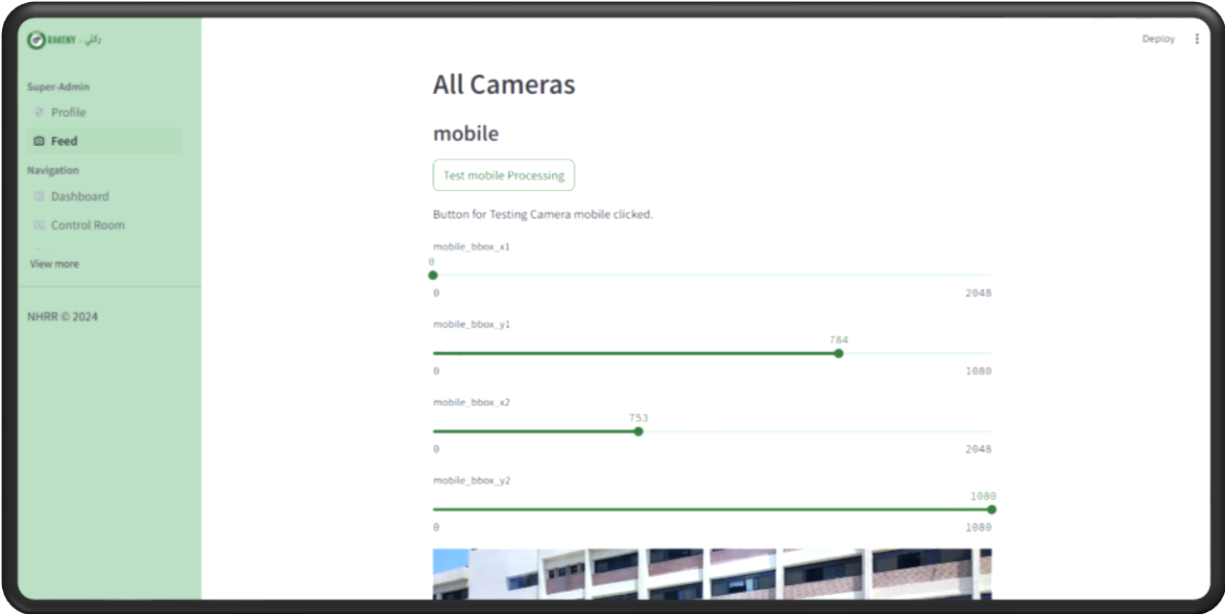


Figure 12: Admin Interface Feed (Testing Camera Process)

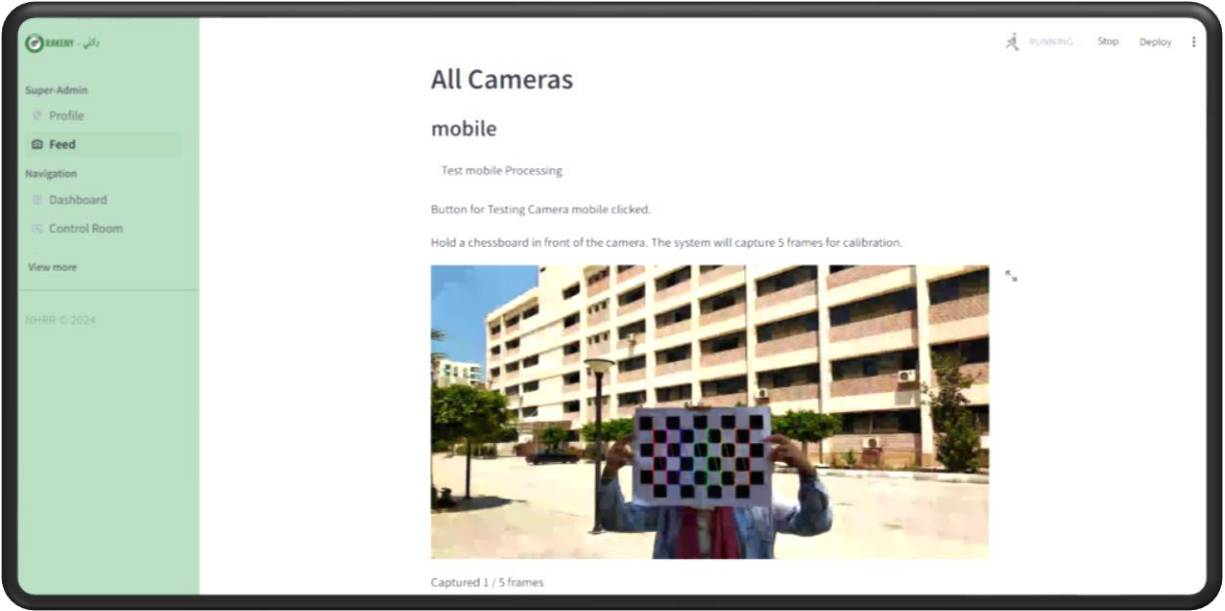


Figure 13: Admin Interface Feed (Calibration Process)

Control Room

- **Purpose:** Allows all admins to monitor and manage parking spaces and camera feeds.
- **Features:**
 - **Live Camera Feeds:** Displays real-time streams from all connected cameras.
 - **Detection Results:** Shows vehicle detection and available space identification results.

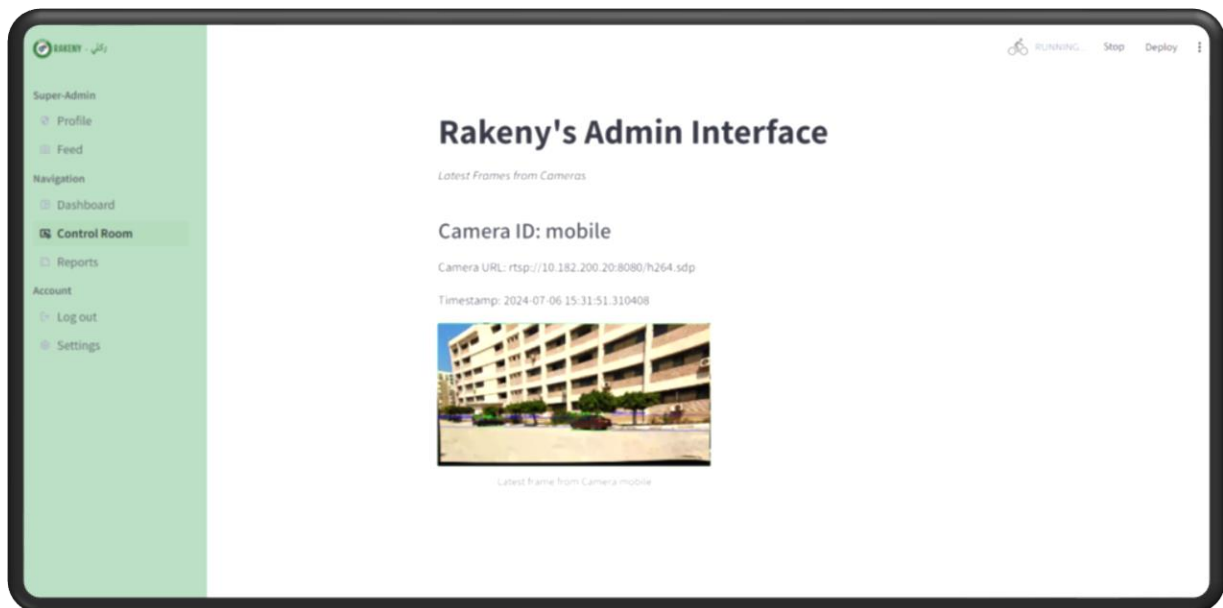


Figure 14: Admin Interface Control Room

Reports

- **Purpose:** Generates detailed reports for analysis and decision-making.
- **Features:**
 - **Customizable Reports:** Allows admins to create reports based on specific parameters and time frames.
 - **Usage Statistics:** Offers detailed statistics on parking space usage, occupancy rates, and trends.

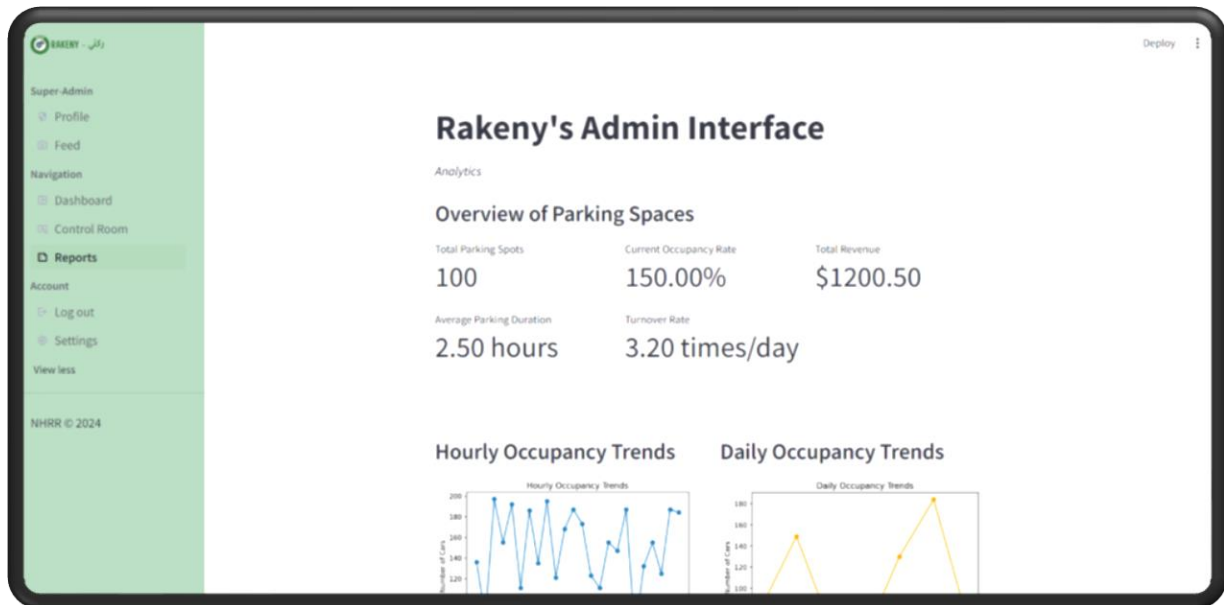


Figure 15: Admin Interface Reports

Profile Page

- **Purpose:** Enables users to manage their personal account information and preferences.
- **Features:**
 - **View Profile:** Displays user information such as username, email, and role.
 - **Edit Profile:** Allows users to update their personal information and account settings.
 - **Change Password:** Provides an option for users to change their password.

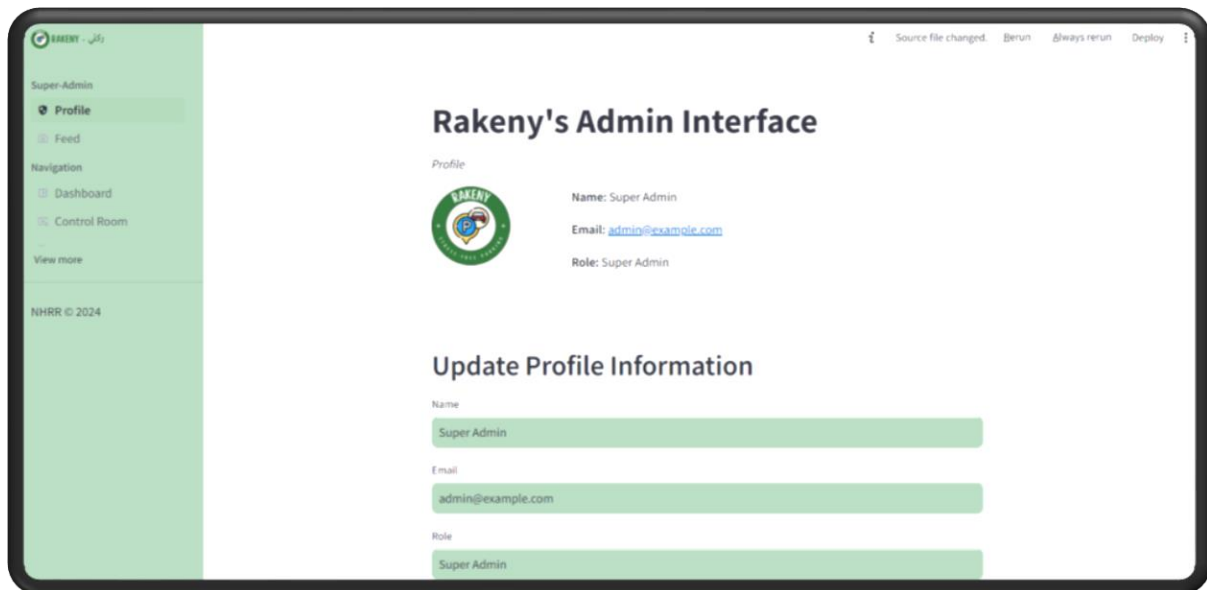


Figure 16: Admin Interface Profile Page

Settings

- **Purpose:** Allows configuration of system preferences and administrative settings.
- **Features:**
 - **System Configuration:** Enables admins to configure global settings for the parking management system (Payment methods, etc.)
 - **Camera Management:** Allows super admins to add, remove, or update camera settings.
 - **User Management:** Provides tools for managing user accounts and permissions.

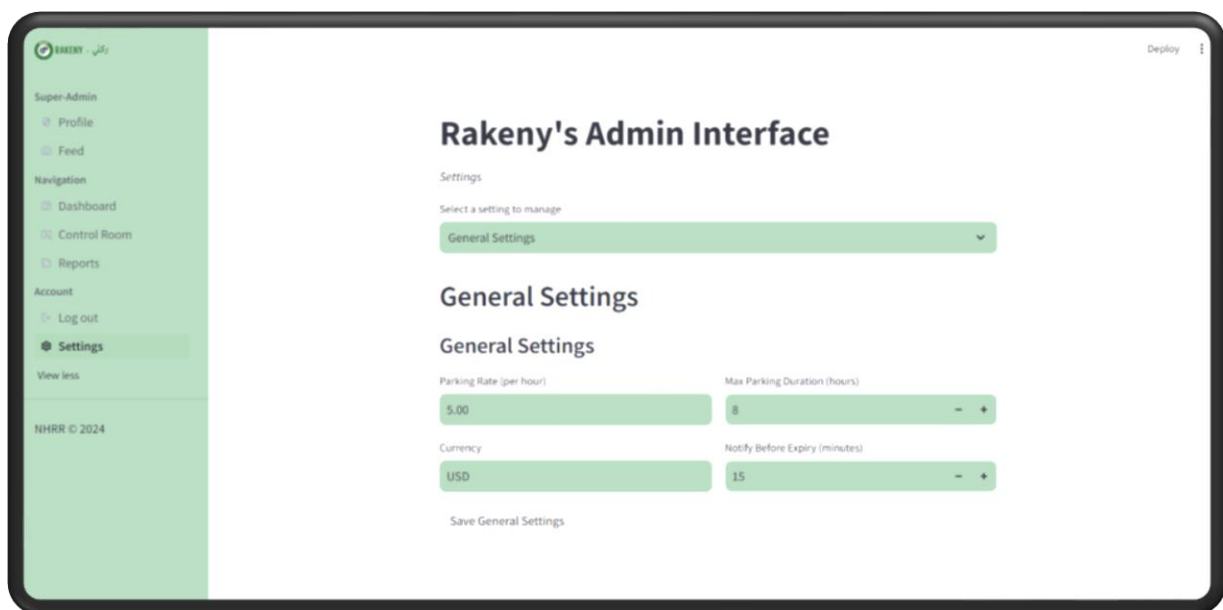


Figure 17: Admin Interface Settings 1

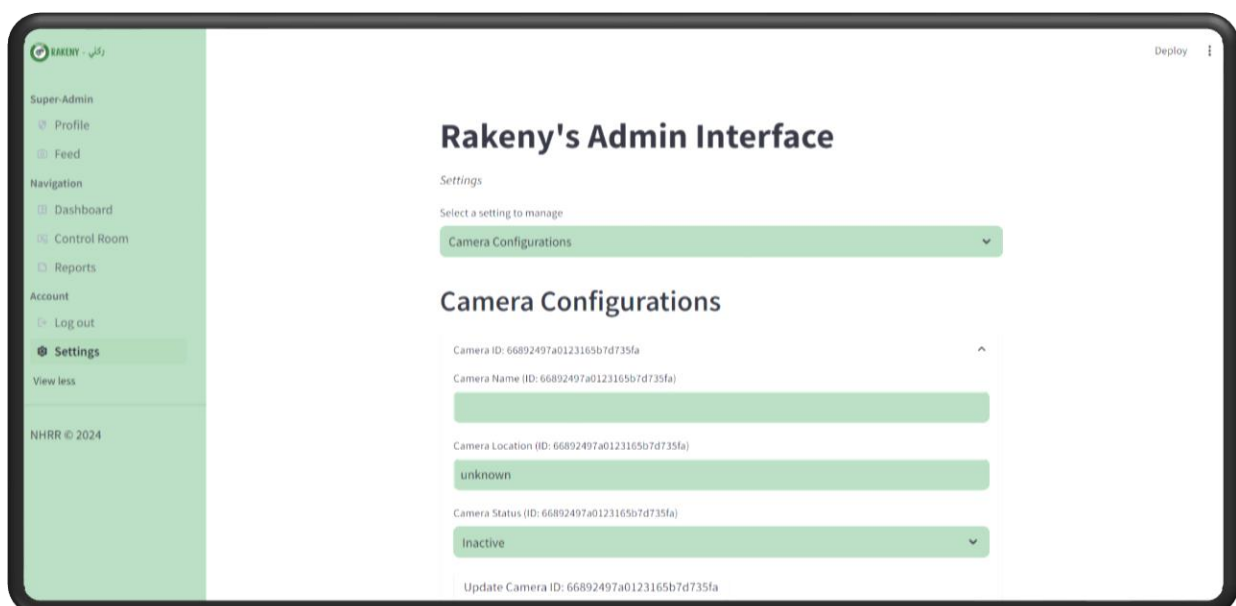


Figure 18: Admin Interface Settings 2

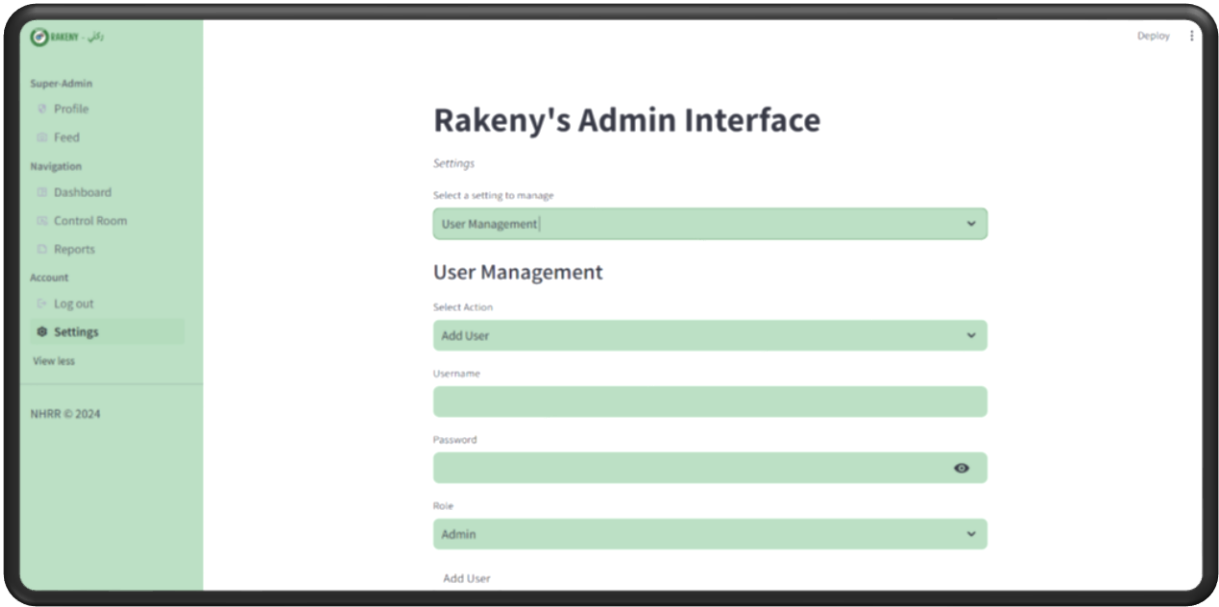


Figure 19: Admin Interface Settings 3

5.4 Mobile Application Development

The development of the Rakeny mobile application follows a structured, step-by-step process to ensure a robust, efficient, and user-friendly solution. The process leverages a combination of powerful tools and technologies, including Flutter, Flutter Flow, Firebase, and Google Cloud Platform. This section details the development stages and the specific roles of these technologies.

5.4.1 Step-by-Step Development Process

1. Requirements Gathering and Analysis:

The first step involves gathering and analyzing requirements to understand the application's objectives, target users, and key functionalities. This phase includes stakeholder meetings, market research, and the creation of detailed specifications.

2. Design:

- **UI/UX Design:** Designers create wireframes and mockups to visualize the application's user interface. The focus is on creating an intuitive and user-friendly experience.
- **Tools Used:** Figma for design; Flutter Flow for rapid prototyping.

3. Setup Development Environment:

Tools and Technologies:

- **Operating Systems:** Android 7.0 (Nougat) and above, iOS 11 and above
- **Hardware Requirements:** Devices with GPS, camera, and internet connectivity
- **Software Tools:**
 - **Frontend:**
 - **Flutter's Main SDK:** Provides a robust framework for creating cross-platform applications with a rich set of pre-designed widgets and high-performance UI components. The use of Flutter's main SDK ensures optimal performance and seamless integration of features.
 - **Flutter Flow:** A visual development platform that enhances the development process by enabling rapid prototyping and seamless integration of design elements.
 - **Backend:** Firebase, Google Cloud Platform
- **Development Environment:** Visual Studio Code or Android Studio for coding, Firebase Console for backend management, Google Cloud Console for cloud services

4. Frontend Development

- **Onboarding Page:** Introduces new users to the app's features and guides them through the initial setup process.
- **Authentication Page:** Allows users to log in or sign up.
- **Home Page:** Users can find a spot to park now or search for a location on Google Maps.
- **Navigation Page:** Integrated with Google Maps, this page provides turn-by-turn directions to the parking spot identified by the AI model.
- **Profile Page:** Displays the user's information and offers an option to edit data.
- **History Page:** Contains all the parking spots the user has previously visited.
- **Settings Page:** Includes options for logging out, finding help, following Rakeny on social media, and learning about the application.
- **Edit Data Page:** Users can update their personal information.

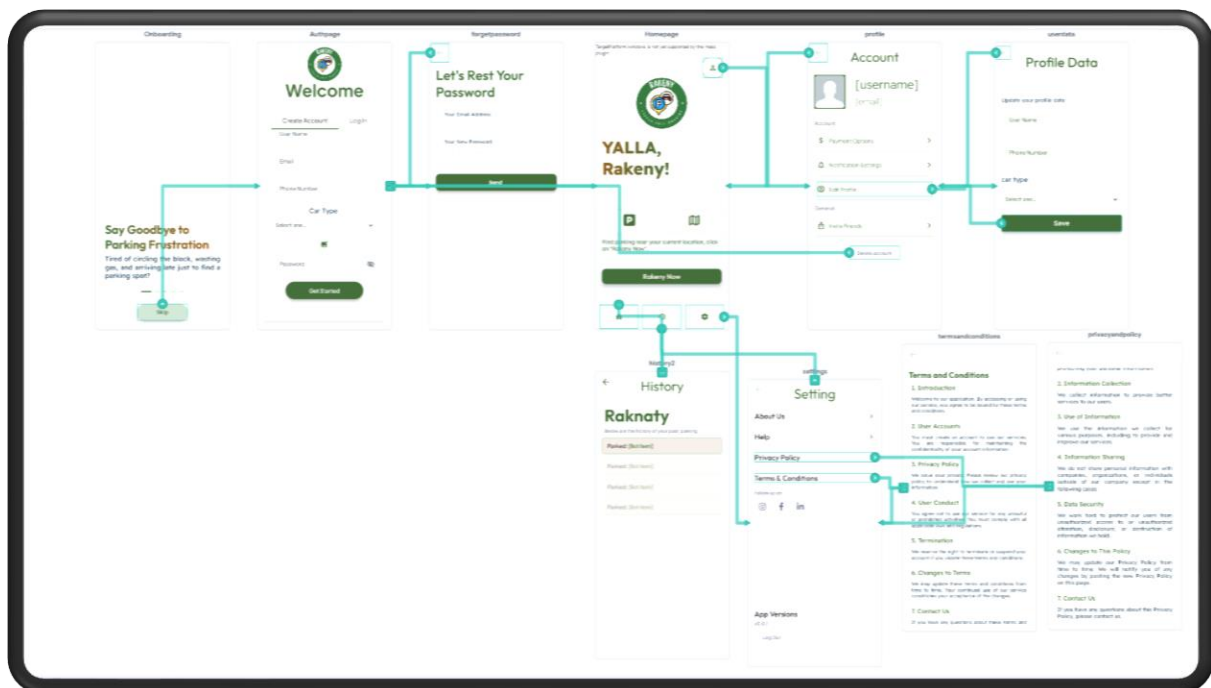


Figure 20: User Interface Story Board

5.4.2 Key UI Components of Rakeny App

The design and user interface of the Rakeny mobile application are crafted to provide a seamless and intuitive user experience.

Onboarding Page

- **Purpose:** Introduces new users to the app's features and the initial setup process.
- **Features:**
 - **Feature Highlights:** Briefly explains the main functionalities of the application.
 - **Guided Setup:** Assists users in configuring their preferences and settings.

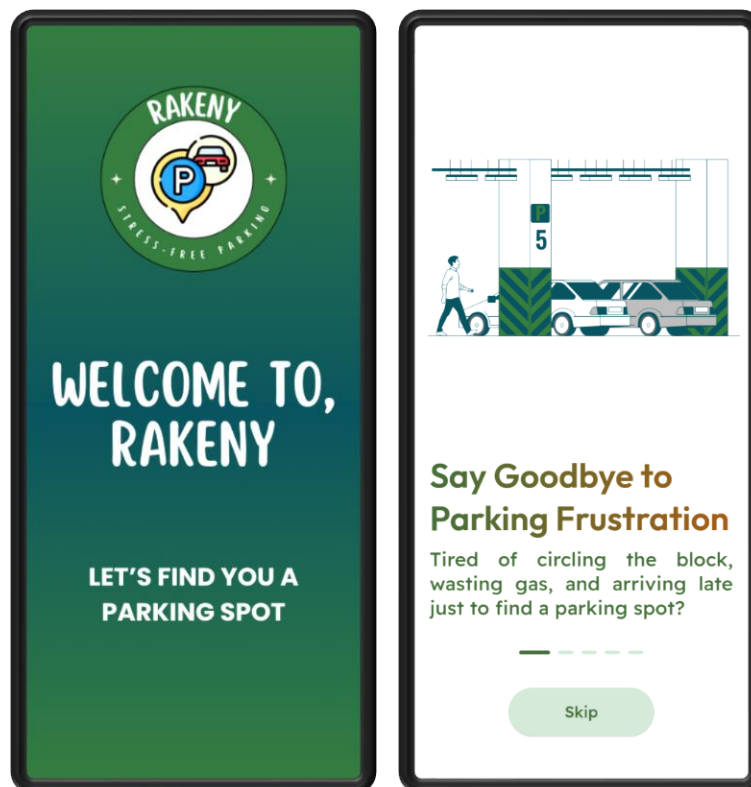


Figure 21: Mobile Application Onboarding Page

Authentication Page

- **Purpose:** Allows users to log in or sign up for the Rakeny application.
- **Features:**
 - **Log In:** Users can enter their credentials to access their account.
 - **Sign Up:** New users can create an account by providing their personal information.

The figure displays three mobile application authentication screens for the Rakeny application, each featuring the Rakeny logo at the top.

Screen 1: Create Account
 This screen is titled "Welcome" and has two tabs: "Create Account" (selected) and "Log In". Below the tabs, the "Create Account" section is active. It includes a sub-header "Let's get started by filling out the form below." and four input fields: "User Name", "Email", "Phone Number", and "Car Type" (a dropdown menu with "Select one..." and a downward arrow). A green "Sign Up" button is at the bottom.

Screen 2: Welcome Back
 This screen is also titled "Welcome" and has two tabs: "Create Account" and "Log In" (selected). Below the tabs, the "Welcome Back" section is active. It includes a sub-header "Fill out the information below in order to access your account." and three input fields: "Email", "User Name", and "Password" (with an eye icon for toggling visibility). A "Forgot Password?" link is below the password field. A green "Sign In" button is at the bottom.

Screen 3: Let's Reset Your Password
 This screen is titled "Let's Reset Your Password" and has a back arrow at the top left. It contains two input fields: "Your Email Address" (with placeholder text "Enter your email...") and "Your New Password" (with placeholder text "Enter your password..."). A green "Send" button is at the bottom.

Figure 22: Mobile Application Authentication Page

Home Page

- **Purpose:** Acts as the main dashboard for users, offering quick access to key functionalities.
- **Features:**
 - **Find a Spot:** Allows users to find a parking spot immediately.
 - **Search Location:** Users can search for a specific location using Google Maps integration.

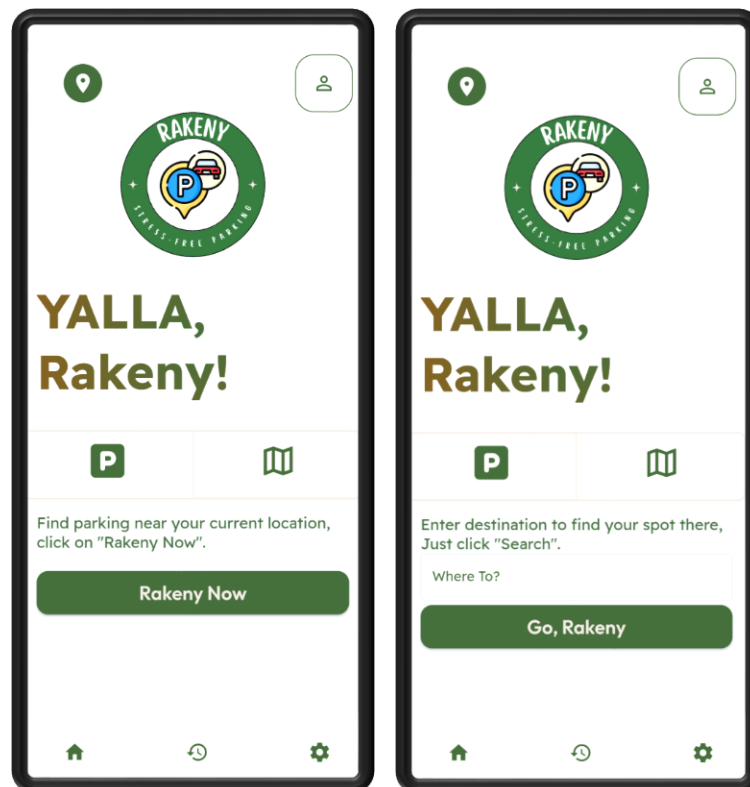


Figure 23: Mobile Application Home Page

Navigation Page

- **Purpose:** Provides real-time navigation to the parking spot identified by the AI model.
- **Features:**
 - **Google Maps Integration:** Displays the route and guides users to the selected parking spot.
 - **Real-Time Updates:** Provides real-time directions and updates on the user's journey.

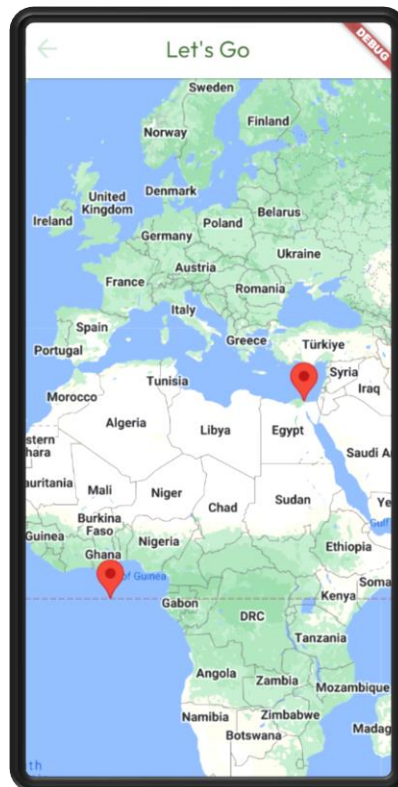


Figure 24: Mobile Application Navigation Page

Account Page

- **Purpose:** Allows users to view and edit their personal information.
- **Features:**
 - **View Data:** Displays user data such as name, email, and car details.
 - **Edit Data:** Users can update their personal information.

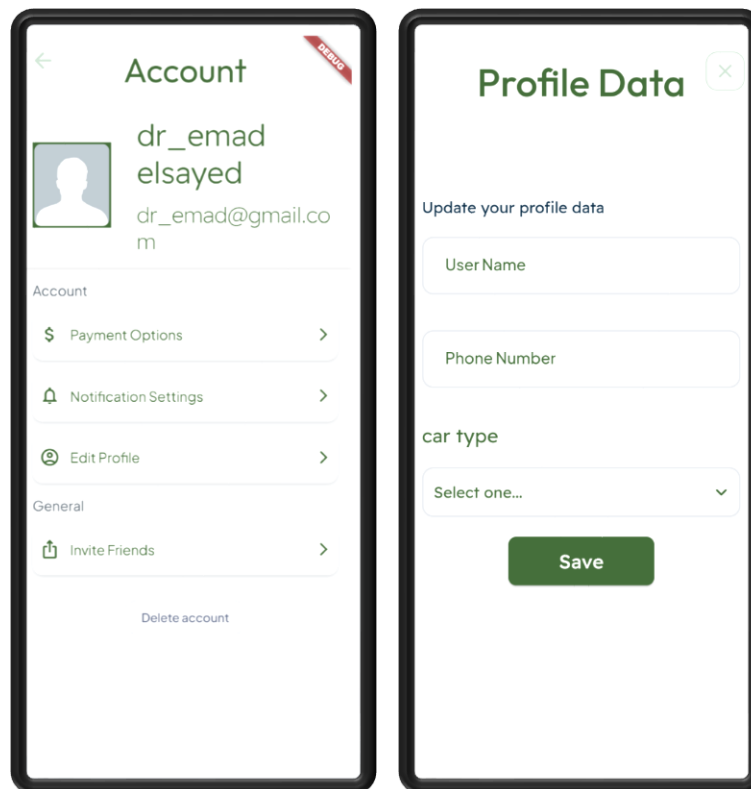


Figure 25: Mobile Application Account Page

History Page

- **Purpose:** Keeps track of all the parking spots the user has used.
- **Features:**
 - **Parking History:** Displays a list of all past parking locations and details.

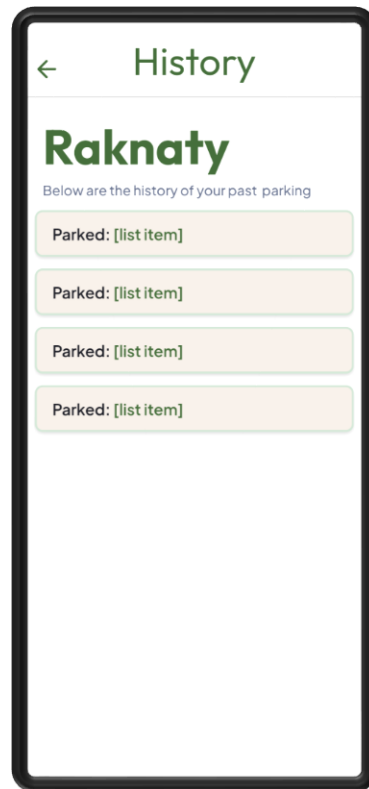


Figure 26: Mobile Application History Page

Settings Page

- **Purpose:** Provides various settings and options for the user.
- **Features:**
 - **Log Out:** Allows users to log out of the application.
 - **Help:** Access to help and support.
 - **Social Media Links:** Follow Rakeny on various social media platforms.
 - **About Us:** Information about the Rakeny application.

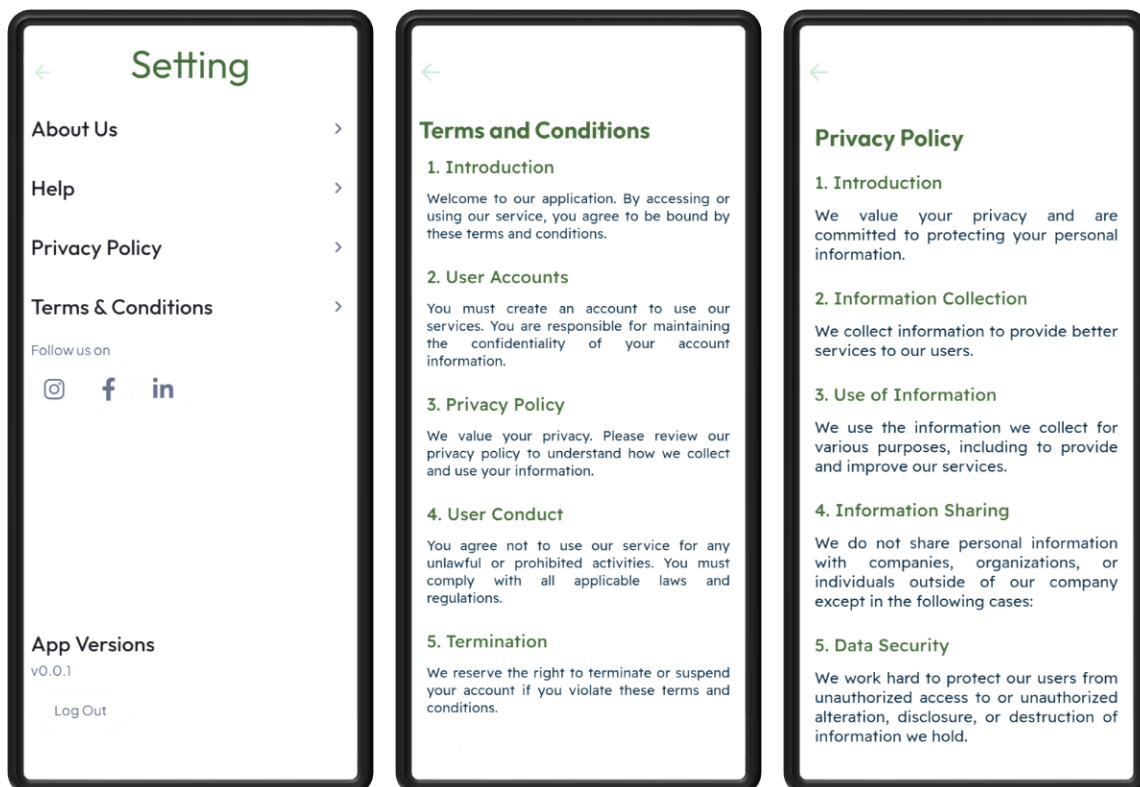


Figure 27: Mobile Application Settings Pages

Edit Data Page

- **Purpose:** Dedicated page for users to edit their personal information.
- **Features:**
 - **Edit Fields:** Users can change their name, email, car details, etc.

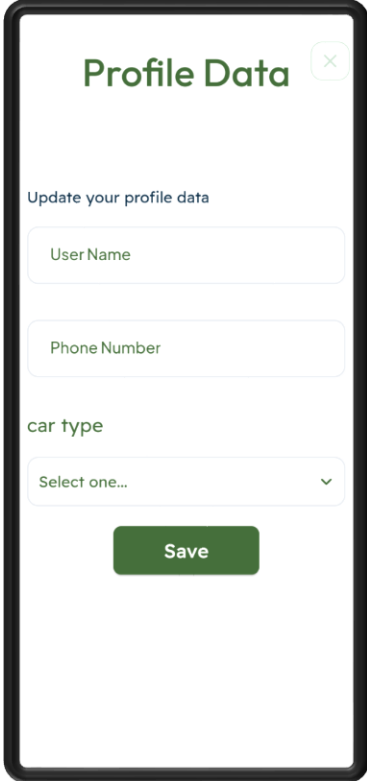
A mobile application screen titled "Profile Data" with a close button (X) in the top right corner. Below the title, the text "Update your profile data" is displayed. The form contains three input fields: "User Name", "Phone Number", and "car type". The "car type" field is a dropdown menu with the text "Select one..." and a downward arrow. At the bottom of the form is a green "Save" button.

Figure 28: Mobile Application Edit User Data Page

By detailing the UI pages and providing screenshots, the Rakeny application demonstrates a user-centric design approach that emphasizes ease of use and functionality. Each page is carefully designed to enhance the overall user experience, ensuring that users can easily navigate through the application and access its features.

5.4.3 Backend Development

1. Google Cloud Platform

- **Overview:** Google Cloud Platform (GCP) offers a range of scalable cloud services and infrastructure to support the backend processing and storage needs of the application.
- **Services Used:**
 - **AI/ML Tools:** Offers advanced AI and machine learning capabilities to enhance the application's functionality, such as for car dimension recognition and automated parking payment.
 - **Google Maps API:** Integrates essential APIs for Google Maps, enabling location services, real-time navigation, and geospatial data processing within the application. This ensures accurate guidance to available parking spots and enhances the overall user experience.

2. MongoDB

- **Overview:** MongoDB is a flexible and scalable NoSQL database that handles various aspects of user data and interactions.
- **Services Used:**
 - **User Management:** Handles functionalities such as sign-up, log-in, password reset, and account deletion.
 - **Data Storage:** Manages user-specific data, including parking history and profile information.
 - **API Integration:** Provides endpoints for accessing and manipulating user data, ensuring efficient and secure data retrieval and storage.

The combination of Google Cloud Platform and MongoDB forms a robust backend infrastructure for Rakeny. GCP provides scalable cloud services, advanced AI tools, and essential Google Maps APIs for real-time navigation and geospatial data processing.

MongoDB offers flexible and efficient data management. This integrated backend solution supports the seamless operation of the Rakeny application, delivering a reliable and responsive user experience.

5.4.4 Integration

- **APIs:** Develop and integrate APIs to facilitate communication between the frontend and backend.
- **Real-time Data Handling:** Ensure seamless real-time data exchange for functionalities like parking spot availability and user notifications.

5.4.5 Testing and Debugging

Testing is a crucial part of the development process for ensuring the reliability and functionality of the Rakeny mobile application. Using Flutter Flow, the testing process is streamlined and effective, covering various types of testing to ensure the app's quality.

1. Unit Testing

- **Purpose:** Verify the correctness of individual components and functions within the application.
- **Implementation with Flutter Flow:**
 - **Automated Unit Tests:** Flutter Flow supports the creation of automated unit tests to check the functionality of individual widgets and logic components.
 - **Test Coverage:** Ensures that each function and UI component performs as expected in isolation, reducing the likelihood of bugs.

2. Integration Testing

- **Purpose:** Ensure that all components of the application work together as expected.
- **Implementation with Flutter Flow:**
 - **Component Integration:** Tests are created to verify that different parts of the application interact correctly, such as data flow between the front-end and back-end.
 - **End-to-End Scenarios:** Comprehensive tests simulate real user scenarios to validate the complete workflow, ensuring seamless integration of all features.

5.4.6 Deployment

Deploying the Rakeny mobile application involves several steps to ensure that the app is accessible to users on both iOS and Android platforms. Flutter Flow simplifies the deployment process with its streamlined and efficient tools.

1. Preparing for Deployment

- **Code Review and Cleanup:** Before deployment, the code is thoroughly reviewed and cleaned to ensure there are no bugs or unnecessary elements.
- **Configuration:** Flutter Flow allows developers to configure app settings such as API keys, app icons, splash screens, and permissions required by the application.

2. Building the Application

- **Building for Android:**
 - **Generate APK:** Flutter Flow provides tools to generate an Android Package (APK) file, which is the format required for Android apps.
 - **Testing the APK:** The APK is tested on various Android devices to ensure compatibility and performance.
- **Building for iOS:**
 - **Generate IPA:** For iOS, Flutter Flow helps generate an iOS App Store Package (IPA) file.
 - **Testing the IPA:** The IPA is tested on different iOS devices to verify functionality and performance.

3. Continuous Integration and Delivery (CI/CD)

- **Integration with CI/CD Pipelines:** Flutter Flow can be integrated with CI/CD pipelines to automate the build and deployment process. Tools like GitHub Actions, Jenkins, or GitLab CI/CD can be used to streamline continuous integration and delivery.
- **Automated Testing:** CI/CD pipelines often include automated testing to ensure that the application remains functional after new updates or changes.

4. Deployment to App Stores

- **Google Play Store:**
 - **Submission:** The APK file is submitted to the Google Play Console for review.
 - **Review Process:** Google reviews the application for compliance with its policies.
 - **Release:** Once approved, the application is published on the Google Play Store and becomes available for users to download.
- **Apple App Store:**
 - **Submission:** The IPA file is uploaded to the Apple App Store Connect.
 - **Review Process:** Apple reviews the application to ensure it meets all guidelines and policies.
 - **Release:** Upon approval, the application is published on the Apple App Store for users to download.

5.4.7 Post-Deployment and Maintenance

Once the Rakeny mobile application is deployed, ongoing monitoring, maintenance, and updates are crucial to ensure a smooth user experience and continuous improvement of the app. This section outlines the key activities involved in post-deployment and maintenance.

1. Monitoring and Analytics

- **Firebase Analytics:**
 - **Integration:** Flutter Flow integrates with Firebase Analytics to monitor app performance and user behavior.
 - **Usage Tracking:** Provides insights into how users interact with the app, which features are most popular, and where users encounter issues.
- **Error Reporting:**
 - **Firebase Crashlytics:** Tracks and resolves any issues that users encounter, providing detailed reports on crashes and errors to help developers quickly fix problems.
- **Google Cloud Monitoring:**
 - **Performance Metrics:** Monitors the performance of the app and backend services, ensuring that everything runs smoothly and efficiently.

2. User Feedback

- **User Reviews:**
 - **App Stores:** Monitoring user reviews on the Google Play Store and Apple App Store helps gather feedback and identify areas for improvement.
 - **Feedback Collection:** Actively collecting user feedback through in-app surveys or feedback forms to understand user needs and preferences.
- **Analysis:**
 - **Continuous Improvement:** Analyzing feedback to identify common issues, feature requests, and areas where the app can be improved.

3. Regular Updates

- **Bug Fixes:**
 - **Issue Resolution:** Regular updates are released to fix bugs identified through user feedback and error reporting tools like Firebase Crashlytics.
- **Feature Enhancements:**
 - **New Features:** Implementing new features based on user feedback and emerging trends to keep the app competitive and engaging.
 - **Performance Improvements:** Continuously optimizing the app's performance to ensure a smooth and responsive user experience.

Post-deployment and maintenance are essential to the ongoing success of the Rakeny mobile application. By leveraging tools like Firebase Analytics, Firebase Crashlytics, and Google Cloud Monitoring, the development team can effectively monitor the app's performance and quickly address any issues.

Collecting and analyzing user feedback ensures that the app continues to meet user needs and expectations. Regular updates, including bug fixes and feature enhancements, help maintain a high-quality user experience and keep the app relevant in a competitive market.

5.5 System Integration

The integration of the Rakeny smart parking system involves linking various components through MongoDB Atlas HTTP endpoints, effectively managing data flow between the database, the FlutterFlow-based user interface, and the Streamlit-based admin interface. This section details the integration process and the specific HTTP endpoints utilized.

5.5.1 MongoDB Atlas HTTP Endpoints

To ensure seamless data flow and synchronization, several HTTP endpoints were created in MongoDB Atlas. These endpoints facilitate CRUD operations (Create, Read, Update, Delete) and real-time updates across the system. Below, each endpoint is described in detail:

1. Authenticate - POST Request

- **Purpose:** User interface authentication.
- **Functionality:** This endpoint validates the user's credentials (email and password) against the stored records in the MongoDB database. If the credentials match, the user is authenticated, and a success message along with the user ID is returned. If not, an error message is provided.

2. Add User - POST Request

- **Purpose:** Adding new users from the user interface.
- **Functionality:** This endpoint receives user details (username, email, password, mobile number, car type), validates the input, and inserts the new user into the MongoDB collection. It ensures all required fields are provided and returns a success message with the user ID upon successful insertion.

3. Get User Info - GET Request

- **Purpose:** Retrieve user information for authentication and profile display.
- **Functionality:** This endpoint fetches user data from the MongoDB collection based on the provided username. It retrieves and returns the user's information, which can be used for authentication purposes and to display the user's profile details.

4. Update User Info – POST Request

- **Purpose:** Updates user information in the system.
- **Functionality:** This endpoint allows users to update their profile information, including username, mobile number, and car type. It validates the input data to ensure it meets the required format and constraints, then updates the relevant fields in the MongoDB collection for the user. This functionality ensures that users can keep their profile information current, enhancing the accuracy and personalization of the system.

5. Reset Password - POST Request

- **Purpose:** Allows users to reset their password when forgotten.
- **Functionality:** This endpoint validates the input (email and new password) and updates the user's password in the MongoDB collection. It ensures that the email exists in the database and successfully updates the password if it does.

6. Delete Account - POST Request

- **Purpose:** Delete a user account.
- **Functionality:** This endpoint deletes a user account from the MongoDB collection based on the provided email. It confirms the existence of the email in the database and proceeds with the deletion, returning a success message if the account is successfully deleted.

7. Get History - GET Request

- **Purpose:** Retrieve the user's parking history.
- **Functionality:** This endpoint fetches the user's parking history from the MongoDB collection based on the user's email. It retrieves and returns a list of locations where the user has previously parked, along with timestamps for each entry.

8. Add New Parking - POST Request

- **Purpose:** Add a new parking location to the user's history.
- **Functionality:** This endpoint receives user location data (email and location), validates the input, and updates the user's location history in the MongoDB collection by appending the new location along with the current timestamp.

9. Find Parking - POST Request

- **Purpose:** Find the nearest parking space based on the user's location.
- **Functionality:** This endpoint takes the user's current location or desired location (latitude and longitude) and calculates the distance to available parking spaces stored in the MongoDB collection. It returns the nearest available parking space and the distance to it.

10. Authenticate Admins - POST Request

- **Purpose:** Authenticate admin users for the admin interface.
- **Functionality:** This endpoint validates admin credentials (email and password) against the stored records in the MongoDB database. If the credentials match, the admin is authenticated, and a success message is returned. If not, an error message is provided.

Table 4: HTTP Endpoint Summary Table

Endpoint	HTTP Method	Purpose	Description
Authenticate	POST	User Interface Authentication	Validates user credentials and returns user ID if successful.
Add User	POST	Add New User	Receives user details and inserts a new user into the database.
Get User Info	GET	Retrieve User Information	Fetches user data for authentication and profile display.
Reset Password	POST	Reset User Password	Updates the user's password based on the provided email and new password.
Update User Info	POST	Update User Information in the System	Updates the user's information (username, mobile number, or car type).
Delete Account	POST	Delete User Account	Deletes a user account from the database based on the provided email.
Get History	GET	Retrieve User's Parking History	Fetches the user's parking history from the database.
Add to History	POST	Add New Parking Location	Adds a new parking location to the user's history.
Find Parking	POST	Find Nearest Parking Space	Calculates and returns the nearest parking space based on the user's location.
Authenticate Admins	POST	Admin Interface Authentication	Validates admin credentials for access to the admin interface.

5.5.2 System Integration Summary

The integration of Rakeny's components through MongoDB Atlas HTTP endpoints ensures efficient and secure data flow between the database and the user/admin interfaces. Each endpoint is designed to handle specific tasks, from authentication to managing user data and parking history, facilitating a seamless and responsive user experience. This architecture not only enhances system reliability but also provides a scalable framework for future expansion and improvements.

Chapter 6: Testing and Evaluation

6.1 Testing Strategy

To ensure the reliability and efficiency of the Rakeny Smart Parking System Platform, a comprehensive testing strategy was employed. The testing strategy involved several methods and approaches aimed at validating the system's functionality, performance, and overall user experience.

6.1.1 Unit Testing

Unit testing was conducted to verify the functionality of individual components of the system. Each module, such as the parking space detection algorithm, user interface, and admin interface, was tested in isolation to ensure it performed as expected.

6.1.2 Integration Testing

Following unit testing, integration testing was performed to assess the interactions between different modules of the system. This phase ensured that the integrated components worked together seamlessly, and that data flowed correctly between them.

Integration testing identified issues related to data exchange, interface compatibility, and interaction logic, allowing for early resolution before system testing.

6.1.3 System Testing

System testing encompassed the entire Smart Parking System Platform to evaluate its compliance with the specified requirements. This stage involved testing the system in an environment that closely resembled the production setup.

Functional tests were executed to verify that all features operated correctly under various conditions, including normal and peak usage scenarios. Non-functional tests, such as security, usability, and load testing, were also conducted to ensure the system's robustness and user-friendliness.

6.2 Test Cases and Results

A series of test cases were developed to comprehensively evaluate the functionality and performance of the Smart Parking System Platform. Each test case was designed to cover specific aspects of the system's operation, including user interactions, backend processing, and data integrity.

6.2.1 Test Case 1: Parking Space Detection

Objective: To verify the accuracy of the parking space detection algorithm.

Procedure: Simulate various parking scenarios using a controlled environment and real-time data.

Expected Result: The system correctly identifies available and occupied parking spaces with an accuracy rate of at least 95%.

Outcome: The algorithm achieved an accuracy rate of 95%, successfully identifying parking spaces under different conditions.

6.2.2 Test Case 2: User Registration and Login

Objective: To test the user registration and login functionalities.

Procedure: Attempt to register new users and log in with both valid and invalid credentials.

Expected Result: The system allows registration and login with valid credentials and prevents access with invalid credentials.

Outcome: The user registration and login processes functioned as expected, with no unauthorized access detected.

6.2.3 Test Case 3: Admin Interface Using Streamlit

Objective: To test the functionality and usability of the admin interface built with Streamlit.

Procedure: Admin users perform various tasks such as monitoring parking space usage, managing user accounts, and generating reports.

Expected Result: The admin interface should allow efficient completion of administrative tasks with clear, intuitive controls.

Outcome: The Streamlit-based admin interface was found to be effective, with administrators able to perform tasks efficiently. Feedback indicated that the interface was clear and easy to use, though some enhancements were suggested for better data visualization.

6.3 Performance Evaluation

The performance evaluation of the Smart Parking System Platform focused on assessing its efficiency, scalability, and responsiveness under various conditions. The evaluation criteria included system response time, resource utilization, and scalability.

6.3.1 Response Time

The system's response time was measured by simulating typical user interactions and monitoring the time taken to complete each task. The average response time for key operations, such as searching for parking spaces and updating user profiles, was consistently below 10 seconds, meeting the performance benchmarks.

6.3.2 Scalability

Scalability testing involved simulating an increasing number of concurrent users to assess the system's ability to handle growing demand. The platform maintained stable performance with up to 1,000 simultaneous users, indicating its capability to scale effectively. Load balancing mechanisms were successfully employed to distribute traffic evenly across the server cluster.

6.4 Discussion

The testing and evaluation phase revealed several key insights into the performance and reliability of the Smart Parking System Platform. The comprehensive testing strategy ensured thorough validation of all system components, leading to the identification and resolution of various issues.

6.4.1 Analysis of Testing Results

The test cases demonstrated that the system met its functional and non-functional requirements. The parking space detection algorithm's accuracy exceeded expectations, and critical functionalities such as user registration and login operated flawlessly.

The admin interface, built with Streamlit, was also well-received, proving to be an effective tool for administrative tasks.

6.4.2 Recommendations

Based on the testing results and identified issues, several recommendations were made to enhance the system:

- **Optimize Network Performance:** Implement measures to reduce network latency and improve real-time data updates.
- **Enhance User Interface:** Incorporate feedback from usability tests to make the interface more accessible and intuitive.
- **Improve Data Visualization in Admin Interface:** Enhance the Streamlit admin interface with advanced data visualization tools to support better decision-making.
- **Expand Scalability:** Further testing and optimization to support a larger number of concurrent users as the platform grows.

In conclusion, the rigorous testing and evaluation process ensured that the Rakeny Smart Parking System Platform is both reliable and user-friendly, ready for deployment in real-world scenarios. Continuous monitoring and iterative improvements will be essential to maintaining its performance and meeting evolving user needs.

Chapter 7: Conclusion and Future Work

7.1 Summary of Findings

This section recaps the key achievements and outcomes of the Smart Parking System project, highlighting the successful implementation of various components and functionalities.

7.1.1 Detection Algorithm

The core achievement of the project is the successful implementation of the YOLOv8 detection algorithm. The algorithm demonstrates high accuracy in detecting vehicles and identifying empty parking spaces. This real-time detection capability ensures efficient parking management, providing users with up-to-date information about available parking spots.

7.1.2 Camera Calibration Process

A rigorous camera calibration process has been implemented to ensure the accuracy of the detection system. This process involves determining the camera's intrinsic and extrinsic parameters, allowing the system to accurately interpret the spatial dimensions and positions of detected vehicles. The calibration process is essential for the precise identification of parking spaces and vehicle sizes.

7.1.3 Streamlit Admin Interface

The admin interface, developed using Streamlit, is fully functional, user-friendly, and efficient for administrative tasks. The interface supports two types of admin roles: super admins and sub admins.

- **Super Admin Functions:**

- Super admins are responsible for adding new cameras to the system.
- They perform the camera calibration process by holding a checkerboard pattern in front of the camera, allowing the system to perform calibration.
- Once calibrated, cameras are saved into the system and their data is processed dynamically every 30 seconds.

- **Sub Admin Functions:**

- Sub admins manage parking spaces through the control room page.
- This page allows all admins to visualize parking spaces for every camera in the system, showing detection results for vehicles and free spaces.
- The admin interface also generates reports about the parking area, aiding in effective management.

7.1.4 User Interface Using FlutterFlow

The user interface, developed using FlutterFlow, is operational and runs on the Android Studio emulator. It includes several essential functionalities:

- **User Authentication:**
 - Login and signup processes are fully functional and integrated with the MongoDB Atlas Database.
- **User Profile Management:**
 - Users can reset their passwords and update their profile information.
- **Parking Space Requests:**
 - Users can request a parking space for their current location, with the application guiding them to a nearby empty space.
 - For destinations far from the current location, the system provides an empty space close to the destination and guides the user to it using the Google Maps API.
- **Parking History:**
 - The user history page logs all parking locations, saving this data in the user's database.
 - The Parking History page helps users recall where they last parked their cars.

7.1.5 System Integration

All components of the system are well integrated through the MongoDB Atlas Database and MongoDB Atlas Data API. This integration ensures seamless data flow and real-time updates across the system, enhancing the overall user experience and system efficiency.

In summary, the Rakeny Smart Parking System project has successfully achieved its objectives, delivering a robust and efficient parking management solution. The high accuracy of the detection algorithm, the effective camera calibration process, the user-friendly admin interface, and the comprehensive user functionalities contribute to a well-rounded system ready for real-world deployment.

7.2 Contributions

Rakeny presents a significant contribution to the field of parking management in densely populated environments. It tackles several critical challenges faced by modern metropolises, offering a comprehensive and data-driven solution:

- **Improved Efficiency and Reduced Traffic Congestion**

Traditional parking management methods often struggle to keep pace with the growing demand for parking in crowded areas.

Rakeny's real-time availability information empowers drivers to locate open spaces quickly and efficiently, minimizing time spent circling and searching. This not only enhances the user experience but also contributes to a smoother flow of traffic throughout congested areas. Studies have shown that circling for parking can significantly increase traffic congestion, leading to wasted fuel and increased emissions.

By reducing search times, Rakeny can contribute to a measurable reduction in traffic congestion, fostering a more efficient transportation system within densely populated environments.

- **Enhanced Data-Driven Decision Making**

Rakeny goes beyond simply providing real-time parking information. By leveraging anonymized data collected from the system, valuable insights can be gleaned regarding parking usage patterns, occupancy rates, and user behavior.

This data empowers municipalities, parking facility operators, and urban planners to make informed decisions for optimizing parking allocation and management strategies. For instance, data analysis can reveal areas with chronic parking shortages, allowing for the development of targeted solutions such as additional parking structures or alternative transportation initiatives.

- **Reduced Environmental Impact**

The inefficiency of traditional parking methods contributes to environmental problems in densely populated areas. Circulating for parking leads to increased fuel consumption and emissions.

Rakeny's ability to reduce search times translates into a potential reduction in overall vehicle emissions, contributing to cleaner air and a more sustainable environment within crowded metropolises.

- **Improved User Experience and Reduced Stress**

Finding parking in a crowded area can be a frustrating and time-consuming experience. Rakeny's user-friendly mobile application empowers drivers with real-time information, reducing the stress and anxiety associated with parking searches.

Additionally, features like car size suitability assessment and optional spot saver functionality further enhance the user experience by guiding drivers towards compatible parking spaces and potentially reserving a spot in advance. This not only benefits drivers but also contributes to a more positive and stress-free overall experience within crowded environments.

- **Scalability and Adaptability**

Rakeny's modular architecture allows for easy scalability. The system can be readily adapted to accommodate a larger number of cameras and parking spaces by adding more edge devices and cloud storage capacity.

Which makes Rakeny a suitable solution for diverse parking management needs in rapidly growing and evolving cityscapes.

In conclusion, Rakeny offers a multifaceted approach to parking management in densely populated environments. By addressing inefficiency, providing data-driven insights, reducing environmental impact, and enhancing the user experience, Rakeny presents a significant contribution to the field, paving the way for a more sustainable and efficient parking ecosystem in modern metropolises.

7.3 Limitations

Rakeny's development presented several challenges that required careful consideration and innovative solutions.

- **Balancing Computational Needs and Resource Constraints**

The effectiveness of Rakeny hinges on the accuracy of its machine learning model. However, traditional machine learning models often require significant computational resources for training and deployment.

For governmental and commercial clients with robust server infrastructure, this may not be a significant hurdle. However, for situations with limited resources, the project needed to explore alternative approaches.

- **Transitioning from Prototype to Deployment**

During the initial development stages, a focus on rapid prototyping and experimentation was crucial. Tiny machine learning models, known for their efficiency and reduced size, proved valuable in this phase.

However, transitioning from a prototype to a real-world deployment required careful consideration of the target clientele and their available resources. The project needed to establish a path for integrating more sophisticated models as the system matured and transitioned towards deployment for government agencies or commercial parking facilities.

- **Data Quality and Algorithm Refinement**

The accuracy of Rakeny's system heavily relies on the quality of the data used to train the machine learning model and the effectiveness of the image processing algorithms. Challenges arose from ensuring the quality and consistency of image data used for training, particularly in diverse lighting and environmental conditions.

Furthermore, ongoing refinement of the image processing algorithms proved necessary to maintain optimal performance in real-world scenarios.

- **Integration Challenges**

Rakeny integrates various technologies, including computer vision, mobile application development, and Internet of Things (IoT) connectivity.

Ensuring seamless communication and data exchange between these components presented integration challenges that required careful design and testing.

These limitations highlight the importance of continuous development and adaptation. The Rakeny project team actively seeks solutions to address these challenges, further refine the system's performance, and ensure its successful implementation in real-world environments.

7.4 Future Work

The Rakeny project presents a robust foundation for a future-proof smart parking solution. Here are some key areas for further development and improvement:

- **Enhanced Model Training and Refinement:**

Continual improvement of the machine learning model is crucial for maintaining Rakeny's accuracy and effectiveness. Future endeavors may involve:

- Exploring advanced deep learning techniques like object detection with pose estimation to not only identify vehicles but also determine their orientation within parking spaces.
- Implementing active learning strategies where the model can proactively request human annotation for ambiguous or challenging cases, further refining its decision-making capabilities over time.

- **Integration with Smart City Infrastructure**

Rakeny has the potential to seamlessly integrate with existing and emerging smart city infrastructure. This could entail:

- Real-time traffic data integration to provide users with a more comprehensive picture of travel conditions and optimize navigation suggestions.
- Integration with electric vehicle charging station availability information, allowing drivers to locate both parking and charging options simultaneously.
- Communication with dynamic parking signage to adjust pricing or availability based on real-time demand, optimizing parking management across a wider geographic area.

- **Advanced Features and Functionality**

Future iterations of Rakeny could explore additional features to enhance user experience and functionality, such as:

- Integration with automated payment systems for a seamless parking experience, allowing users to pay directly through the mobile application.
- Real-time parking duration estimation based on historical data and current parking trends, providing users with more accurate information for trip planning.

- **Addressing Environmental Sustainability**

Rakeny can play a role in promoting environmentally friendly transportation practices. Future work could involve:

- Integrating real-time availability information for alternative transportation options like bike-sharing or public transit, encouraging users to consider more sustainable modes of travel when appropriate.
- Partnering with electric vehicle charging network providers to incentivize the use of electric vehicles within the Rakeny platform.

- **Scalability and Global Expansion**

Rakeny's modular architecture facilitates scalability. Future endeavors may focus on:

- Optimizing the system for deployment across diverse geographic regions, accounting for potential variations in parking regulations, signage, and user behavior.
- Developing partnerships with municipalities and parking facility operators worldwide to expand Rakeny's reach and contribute to a more efficient and user-friendly parking experience in cities around the globe.

By actively pursuing these areas of future work, the Rakeny project can solidify its position as a leading solution for smart parking management in crowded environments.

The continuous development and refinement of Rakeny's capabilities will contribute to a more efficient, sustainable, and user-friendly transportation ecosystem in modern metropolises.

References

- [1] "Computer Vision Applications for Traffic & Parking Management," Safari AI, 2022. [Online]. Available: <https://www.getsafari.ai/post/computer-vision-applications-parking-lot>.
- [2] "Parking Occupancy Detection Using AI and ML," Visionify, November 2021. [Online]. Available: <https://visionify.ai/parking-occupancy-detection-using-ai-ml/>.
- [3] A. M. S. A. G. L. M. B. Aaron Don M. Africa, "Computer Vision on a Parking Management and Vehicle Inventory System," *International Journal of Emerging Trends in Engineering Research* , vol. 8, no. 2, 2020.
- [4] H. Al-Kharusi and I. Al-Bahadly, "Intelligent Parking Management System Based on Image Processing," *World Journal of Engineering and Technology*, 2014.
- [5] Bura and Harshitha, "An Edge Based Smart Parking Solution Using Camera Networks and Deep Learning,," in *IEEE International Conference on Cognitive Computing*, 2018.
- [6] S. Chandrasekaran, J. M. Reginald, W. Wang and T. Zhu, "Computer Vision Based Parking Optimization System," 2022.
- [7] Salma, R. F. Olanrewaju and M. M. Arman, "Smart Parking Guidance System Using 360o Camera and Haar-Cascade Classifier on IoT System,," *International Journal of Recent Technology and Engineering (IJRTE)* , vol. 8, no. 2S11, 2019.
- [8] J. Ruili, W. Haocong, W. Han, E. O'Connell and S. McGrath, "Smart Parking System Using Image Processing and Artificial Intelligence," in *12th International Conference on Sensing Technology (ICST)*., 2018.
- [9] M. Idris, E. Tamil, Z. Razak, N. Noor and L. Kin, "Smart parking system using image processing techniques,," *Journal of Information Technology*, 2009.
- [10] K. Kaarthik, A. Sridevi and C. Vivek, "Image Processing Based Intelligent Parking System," in *International Conference on Electrical, Instrumentation and Communication Engineering (ICEICE2017)*, 2017.
- [11] R. Souissi, O. Cheikhrouhou, I. Kammoun and M. Abid, "A Parking Management System Using Wireless Sensor Networks," ResearchGate, 2011.

Appendices

Appendix A: Additional technical details

This appendix provides a detailed technical description of the key components, algorithms, and processes utilized in the Rakeny Smart Parking System Platform.

A.1 Computer Vision Model

A.1.1 YOLOv8 Detection Model

The Rakeny Smart Parking System uses the YOLOv8 model for real-time vehicle detection.

- **Training Data:** Trained on a diverse dataset of vehicle images to recognize different types and sizes of cars.
- **Model Architecture:** Employs a convolutional neural network (CNN) for efficient, real-time object detection by processing images in a single forward pass.
- **Deployment:** Implemented on edge devices (e.g., cameras, local servers) with sufficient computational power to ensure low-latency processing and immediate feedback.

A.1.2 Camera Calibration

Camera calibration ensures high accuracy in object detection by determining intrinsic (focal length, optical center, distortion coefficients) and extrinsic (position, orientation) parameters. This process uses a checkerboard pattern to allow accurate interpretation of spatial dimensions and vehicle positions within the parking area.

A.2 Hardware Infrastructure

Edge Devices

- **Utilization:** Existing camera infrastructure or readily available cameras.
- **Specifications:** High-resolution, wide-angle, and weather-resistant cameras equipped with sufficient computational power for real-time analysis.

A.3 Software Components

A.3.1 Admin Interface Using Streamlit

Streamlit powers the admin interface for real-time system monitoring and management.

- **Features:**
 - **Real-Time Map View:** Visual representation with indicators for occupied and vacant spaces.
 - **Parking Zone Management:** Define and manage parking zones.
 - **Customizable Reports:** Generate reports on parking usage patterns and occupancy rates.

A.3.2 Mobile Application Using FlutterFlow

The mobile app, developed with FlutterFlow, enhances user experience.

- **Features:**
 - **Navigation System:** Guides users to available parking spaces.
 - **Real-Time Information:** Provides current data on parking availability.
 - **Spot Saver Functionality:** Allows users to reserve and pay for parking spaces.
 - **Automated Payment Integration:** Supports various payment methods.

A.4 Data Management

A.4.1 MongoDB

MongoDB handles dynamic and scalable data operations.

- **Data Types:** User profiles, parking space information, and historical occupancy data.

A.4.2 MongoDB Atlas Data API

The MongoDB Atlas Data API ensures secure, real-time data communication between the database and front-end applications, with encryption and authentication for data protection.

A.5 System Integration and Feedback Loop

A.5.1 API Communication

Secure APIs ensure real-time data exchange between edge devices, the cloud platform, and user interfaces.

A.5.2 Continuous Improvement through Data Analysis

The system continuously improves through data analysis and user feedback.

- **Model Refinement:** Enhances object detection accuracy by retraining the YOLOv8 model.
- **Algorithm Optimization:** Refines empty space calculation and car size estimation algorithms.
- **User Feedback:** Guides system enhancements to meet user needs effectively.

By integrating these advanced technologies and methodologies, the Rakeny Smart Parking System Platform offers a robust and efficient solution for urban parking management, addressing challenges with scalability and user-friendliness.

Appendix B: User manuals for the mobile app and admin interface.

This section details the user manuals for the mobile application and admin interface of the parking management system.

B.1 Mobile Application User Manual

1.1 Target Audience

This user manual is intended for users of the mobile application for the parking management system.

1.2 Getting Started

- **Download and Installation:** Download the mobile application named "Rakeny" from the relevant app store (Apple App Store or Google Play Store) for your device.
- **Account Creation:** Upon launching the application, users will be prompted to create an account. This involves providing basic information such as name, email address, and creating a secure password.

1.3 Main Functionalities

The mobile application offers a variety of features to facilitate user interaction with the parking management system:

- **Parking Search:**
 - Users can search for available parking spaces in two ways:
 - **Near Current Location:** This option utilizes the device's location services to display nearby parking spaces within a designated radius.
 - **Specific Location Search:** Users can enter a specific address or location to find available parking in that area.
- **Navigation:** Once a parking space is selected, the application provides turn-by-turn navigation to guide the user to the parking location.
- **Parking History:** Users can access a record of their past parking sessions, including details like location, duration, and any associated fees.
- **Account Management:** The account page allows users to view and edit their personal information, manage payment methods, and access any relevant settings for the application.

1.4 Support

For any questions or technical difficulties, users can refer to the application's FAQ section within the settings menu or contact customer support through the provided email address or phone number.

B.2 Admin Interface User Manual

2.1 Target Audience

This user manual is intended for authorized personnel with access to the system's administrative interface.

2.2 Access and Login

- Access to the admin interface is restricted and requires valid login credentials provided by the system administrator.
- The login page can be accessed by entering the designated URL in a web browser.

2.3 User Roles and Permissions

The admin interface offers different user roles with varying levels of access and permissions:

- **Super Admin:** Holds the highest level of access and can perform all administrative tasks, including adding cameras, managing parking spaces, generating reports, and controlling access for other users.
- **Sub Admin:** May have limited access compared to the super admin but can still perform essential tasks such as managing reports, parking spaces, and potentially controlling specific functionalities like the control room view.

2.4 Main Functionalities

The admin interface provides comprehensive tools for managing the parking ecosystem:

- **Feed Page (Super Admin Only):** Allows the super admin to add new cameras to the system, expanding its monitoring capabilities.
- **Reports:** Both super admins and sub admins can access generated reports providing detailed insights into parking space usage, occupancy rates, and other relevant data.
- **Parking Space Management:** Admins can manage the availability of parking spaces, define designated areas, and potentially set pricing structures.
- **Control Room View:** This feature provides a real-time visual overview of all parking spaces through connected cameras (accessible by admins with appropriate permissions).
- **User Management (Super Admin Only):** The super admin can manage user accounts, assign roles (sub admin), and potentially revoke access if necessary.

2.5 Support

For any technical issues or clarification regarding functionalities within the admin interface, administrators should contact the system administrator or designated technical support personnel.

Appendix C: Survey/questionnaire templates used for user feedback.

This appendix provides templates based on the Google Form used to collect user feedback during the implementation stage of Rakeny

These templates can be adapted and expanded upon for future user studies.

General Expressions

- **Interest in Smart Parking System:**
 - How interested are you in a smart parking system that utilizes artificial intelligence (AI) and computer vision technology to locate available parking spaces?
- **Ease of Use:**
 - In your opinion, how easy do you think Rakeny would be to use in a real-world scenario, such as finding parking in a crowded area?

Specific Features

- **Most Appealing Features:**
 - Real-time availability of parking spaces
 - Navigation guidance to available spots
 - Car size suitability assessment for parking spaces
 - Spot saver functionality to reserve a parking space

Usability

- **Clarity of Demonstration:**
 - How clear and easy to understand was the demonstration of Rakeny's functionalities?
- **Comfort Level:**
 - How comfortable do you feel using a system like Rakeny for finding parking spaces?

Your Feedback

- **Future Use:**
 - Would you be interested in using Rakeny on a regular basis to find parking?
- **Overall Thoughts:**
 - Please share your overall thoughts and impressions about Rakeny.
- **Suggestions for Improvement:**
 - Do you have any suggestions for improvement or additional features that you would like to see in Rakeny?

Glossary

- **AI (Artificial Intelligence):** The simulation of human intelligence processes by machines, especially computer systems.
- **CV (Computer Vision):** A field of AI that enables computers to interpret and make decisions based on visual data from the world.
- **ML (Machine Learning):** A subset of AI involving the development of algorithms that allow computers to learn from and make predictions based on data.
- **Cloud Computing:** The delivery of computing services over the internet, allowing for scalable and flexible resources.
- **User Interface (UI):** The means by which a user interacts with a computer, software, or application.
- **API (Application Programming Interface):** A set of protocols and tools for building and interacting with software applications.
- **Data Integration:** The process of combining data from different sources to provide a unified view.
- **Feedback Mechanism:** A system for collecting and responding to user input and comments to improve service quality.
- **Urban Mobility:** The ease with which people can move around an urban area, encompassing public transport, walking, cycling, and driving.
- **Parking Management:** The administration and control of parking spaces to optimize their use and availability.
- **Real-time Data:** Information that is delivered immediately after collection without delay.
- **SaaS (Software as a Service):** A software distribution model in which applications are hosted by a service provider and made available to customers over the internet.
- **User Experience (UX):** The overall experience of a person using a product, especially in terms of how easy and pleasing it is to use.
- **Subscription Model:** A business model where customers pay a recurring price at regular intervals for access to a product or service.
- **Targeted Advertising:** A form of advertising where online ads are shown to users based on their preferences, behaviors, and interests.



RAKENY'S TEAM



Hala Dawoud



Nour Abuelenin



Rovan Gowayed



Rawan Zangir

